Carpeta de campo

Índice

1. Presentación del proyecto Pág. 3
2. Organización inicial y análisis del proyecto Pág. 4 a Pág. 9
    - I. Primer UML y organización del proyecto
    - II. Preparación y realización de la entrevista
    - III. Análisis FODA
    - IV. Requerimientos funcionales y no funcionales
    - V. Diagrama de Contexto (DFD Nivel 0)
    - VI. Análisis tecnológico
3. Desarrollo de clases en Java Pág. 10 a Pág. 18
    - I. Desarrollo de la clase Turno
    - II. UML y controladora de Turnos
    - III. Controladora de Medicamentos
    - IV. Controladora de Enfermedades
    - V. Desarrollo del JFrame de Síntoma
4. Desarrollo del Historial Médico Pág. 21 a Pág. 27
    - I. Desarrollo del JFrame Historial Médico
    - II. Funciones de ingresar, mostrar, buscar, eliminar y editar
    - III. Modificaciones en la base de datos
    - IV. Modificaciones en SintomaDAO
    - V. Correcciones en VentanaBienvenida
5. Maqueta de la Clínica Privada Pág. 28 a Pág. 30
    - I. Construcción de la maqueta en Minecraft
    - II. Consultorios de psicología
    - III. Fisioterapia y rehabilitación
    - IV. Maternidad y sala de conferencias
6. Configuración de Redes Pág. 31 a Pág. 34
    - I. Configuración de usuarios en el servidor de correo
    - II. Asignación de nombres de usuario y contraseñas
    - III. Configuración de correos en las computadoras
7. Conexión Java - Web - SQL Pág. 35 a Pág. 38
    - I. Conexión del servidor web
    - II. Corrección de archivos CSS
    - III. Visualización de imágenes
    - IV. Ejecución de la página desde Java
8. Base de datos: Vistas y Triggers Pág. 39 a Pág. 40

I. Creación de vistas de Historial Médico y usuarios
II. Creación de triggers
III. Comprobación en el servidor PHP
9. Incorporación de nuevas funcionalidades Pág. 41
I. Incorporación de la página Perfil
II. Conexión de CSS de Perfil con Java
10. Testeo y correcciones de la aplicación Pág. 42 a Pág. 47
I. Pruebas de funcionamiento de la aplicación
II. Corrección del botón Salir y mensajes
III. Modificación de GestiónTurno y TurnoDAO
IV. Incorporación de botones de navegación
V. Procedimientos de especialidades en SQL

Clínica Privada

Luciano Villarroel

Escuela Técnica N°32 DE 14 “Gral. José de San Martin”, CABA

Proyecto Interdisciplinario

Nahuel Acosta y

Gabriel Abello Tuinyla

CABA, Argentina

15 de Julio de 2026

Clínica Privada

Integrantes:

Luciano Villarroel

Matias Basto

Bautista Miyashiro

Santiago Ugaz

Martina Orlandi

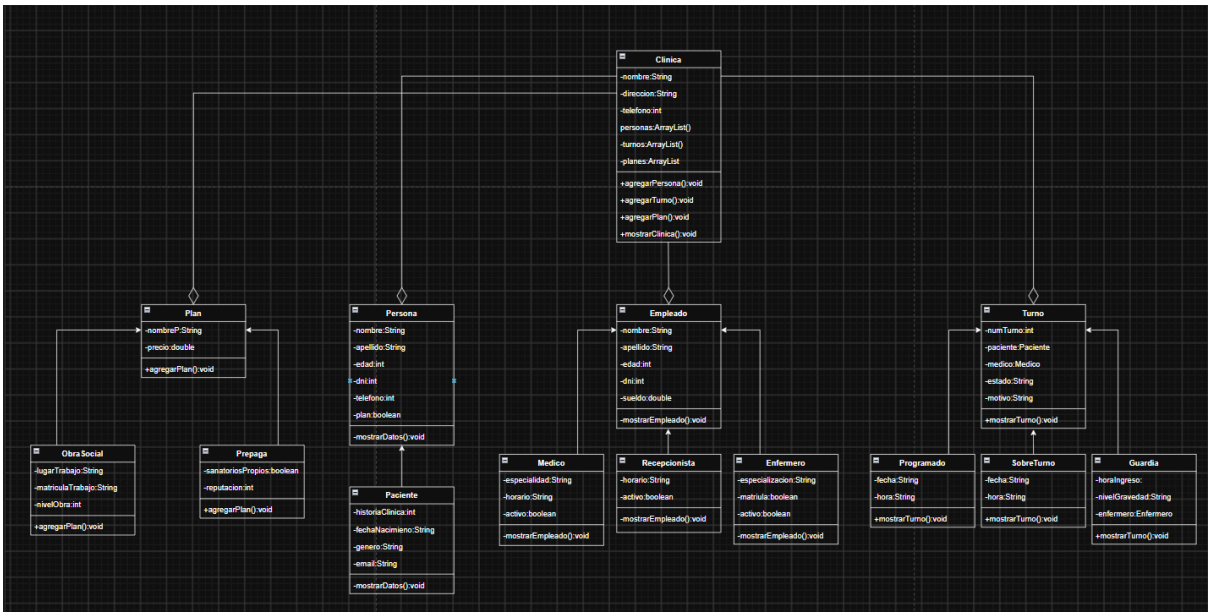
La presente carpeta de campo tiene como objetivo registrar de manera detallada las actividades realizadas por Luciano Villarroel en el marco de las materias: Programación Orientada a Objetos, Redes, Análisis de Sistemas, Base de Datos y Proyecto Informático II. El propósito principal es llevar un control organizado de las tareas, tanto individuales como grupales, a fin de documentar el proceso de trabajo, los avances y las dificultades encontradas.

Los registros incluirán la siguiente información básica en cada entrada: fecha, lugar donde fue realizada la actividad, así como una descripción objetiva de lo que se hizo. Además, se añadirán observaciones o comentarios que permitan reflexionar sobre el desarrollo del trabajo.

Este documento servirá como evidencia del compromiso y participación de Luciano Villarroel, y permitirá evaluar de forma clara el progreso alcanzado durante el período establecido.

15/07

Este día se realizó el primer UML con el grupo, pensando qué atributos debería tener cada cosa. Luego se tuvo que borrar, ya que al no tener la entrevista hecha no servía, porque eran suposiciones de cómo sería y no está fundamentada en alguna necesidad válida de algún usuario.



16/07
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

17/07
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

18/07
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

19/07
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

20/07
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

21/07
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

22/07
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

23/07
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

24/07
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

25/07
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

26/07
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

27/07
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

28/07
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

29/07
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

30/07
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

31/07
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

01/08
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

02/08
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

03/08
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

04/08
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

05/08

Hoy mis compañeros de grupo y yo pasamos nuestros correos electrónicos al grupo de Whatsapp para poder ver un documento, que compartió Santiago Ugaz. Era la problemática planteada y las preguntas principales que se iban a realizar en la entrevista, además de que se iba a hacer.

06/08

Este día no se realizó ninguna tarea/actividad sobre el proyecto, ya que estoy esperando la entrevista para avanzar en el trabajo.

07/08

Este día no se realizó ninguna tarea/actividad sobre el proyecto, ya que estoy esperando la entrevista para avanzar en el trabajo.

08/08

Hoy Santiago Ugaz realizó la entrevista. Cuando la mandó al grupo me puse a leer la entrevista para más o menos tener una idea de cuáles iban a ser las características/fortalezas de la clínica, que es para realizar el análisis FODA.

09/08

El día de la fecha realice el FODA, en base a la entrevista que realizó Cañetes con el padre de un amigo suyo.

Fortalezas ● Analiza valores en tiempo real. ● Unifica la información y criterios para dar una respuesta aproximada. ● Facilita el acceso al historial clínico del paciente. ● Registra medicamentos e información del paciente. ● Genera alertas cuando hay valores anormales.	Oportunidades ● Incorporar inteligencia artificial para mejorar la interpretación de datos. ● Mejorar la continuidad de atención cuando hay cambio de turno o de personal. ● Reducir errores derivados de la falta de comunicación entre el personal médico.
Debilidades ● El software podría interpretar incorrectamente algunos valores. ● Se requiere personal que esté capacitado para interpretar los datos. ● Necesitaría información precisa y actualizada para funcionar correctamente. ● No reemplaza el criterio de un profesional especializado.	Amenazas ● Una recomendación incorrecta. podría generar una responsabilidad legal. ● Problemas de privacidad de los datos médicos de los pacientes. ● Un error del sistema podría afectar negativamente a la atención médica del paciente

10/08

Este día no se realizó ninguna tarea/actividad sobre el proyecto.

11/08

Este día no se realizó ninguna tarea/actividad sobre el proyecto.

12/08
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

13/08
Realice los 25 requerimientos funcionales y no funcionales, en base a la entrevista y a la problemática. A continuación adjunto una parte de los requisitos funcionales y no funcionales.

Funcionales:

1. Registrar nuevos pacientes.
2. Registrar historial clínico.
3. Registrar datos del personal de turno.
4. Que el sistema emita una alerta en una situación anormal.
5. Registrar ingreso y egreso del paciente.

No funcionales:

1. Privacidad de los datos de los pacientes.
2. Las alertas que se emitan deben ser rápidas luego de detectarse la anomalía.
3. Que permita actualizar el sistema sin perder los anteriores datos.
4. Que la interfaz sea intuitiva para el personal médico.
5. Las alertas deben ser claras.

14/08
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

15/08
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

16/08
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

17/08
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

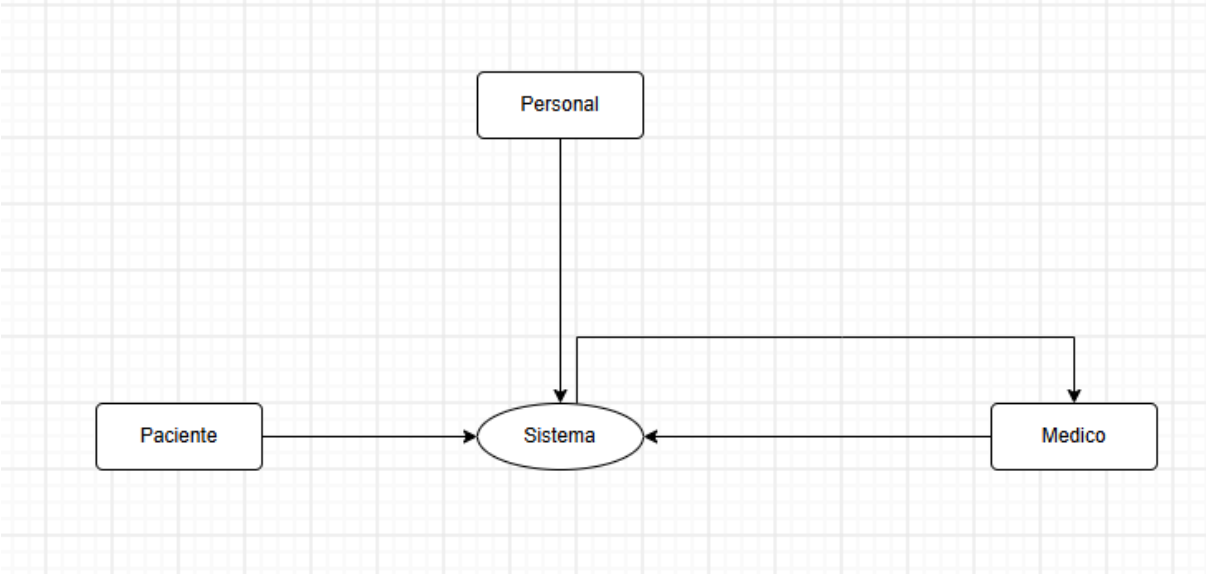
18/08
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

19/08

Este día no se realizó ninguna tarea/actividad sobre el proyecto.

20/08
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

21/08
Comencé a realizar el diagrama de contexto(DFD Nivel 0), solo poniendo los atributos y las flechas en donde irían los flujos, que serían como interactúa dicho atributo/persona con el sistema.



22/08
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

23/08
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

24/08
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

25/08
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

26/08
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

27/08
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

28/08

Empecé a realizar el Análisis tecnológico recopilando las aplicaciones que usamos para hacer cada cosa del proyecto.

Analisis tecnologico

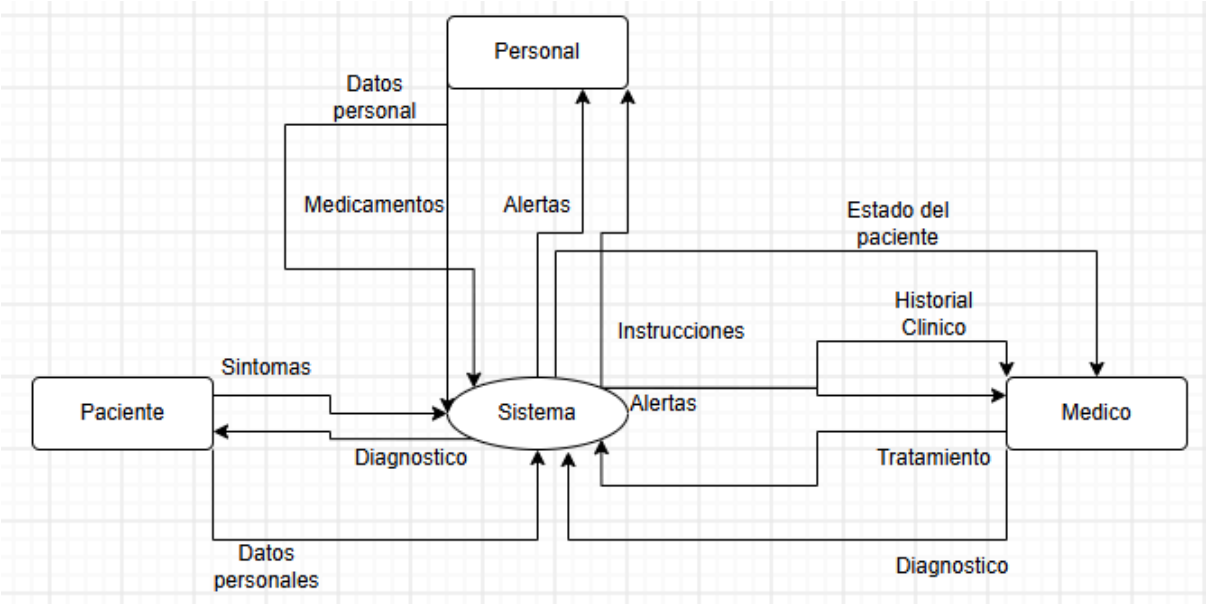
Draw io: Se usa para realizar diagramas. Por ahora lo usamos para realizar el ULM de la clínica basándonos en la entrevista, el CPM y el diagrama de contexto.

Jira: Lo usamos para la asignación de tareas, además de mostrar las tareas que hay que hacer y marcar las tareas completadas.

Github: Se está usando para cargar las actividades hechas.

29/08
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

30/08
Termine el Diagrama de Contexto(DFD nivel 0), agregándole los flujos que correspondían, ya que era lo que me faltaba y agregando un par de atributos más. También termine el análisis tecnológico.



Analisis tecnologico

Draw io: Se usa para realizar diagramas. Por ahora lo usamos para realizar el ULM de la clínica basándonos en la entrevista, el CPM y el diagrama de contexto.

Jira: Lo usamos para la asignación de tareas, además de mostrar las tareas que hay que hacer y marcar las tareas completadas.

Github: Se está usando para cargar las actividades hechas.

Cisco Packet Tracer: En esta aplicación se realizó como quedaría la red de la clínica con su respectiva tipología y cantidad de equipos.

MySQL: Se va a usar para realizar la base de datos de la clínica.

NetBeans: Se usó para realizar el MVP y el código de la clínica, mostrando un menú de selección para ingresar síntomas y para mostrarlos.

Hojas de cálculo: Se usó para hacer el diagrama de Gantt.

Docs: Se usó para realizar el Foda, los requerimientos del sistema y el estado del arte.

31/08
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

01/09
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

02/09
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

03/09
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

04/09
Ese día realice la clase turno en java, y después la agregue al UML la clase turno con sus respectivos atributos y métodos. Además se marcó en el JIRA como completado y se agregó para realizar la tarea de “Añadir enfermedades”.

```java
package javaapplication1;
import javax.swing.JOptionPane;

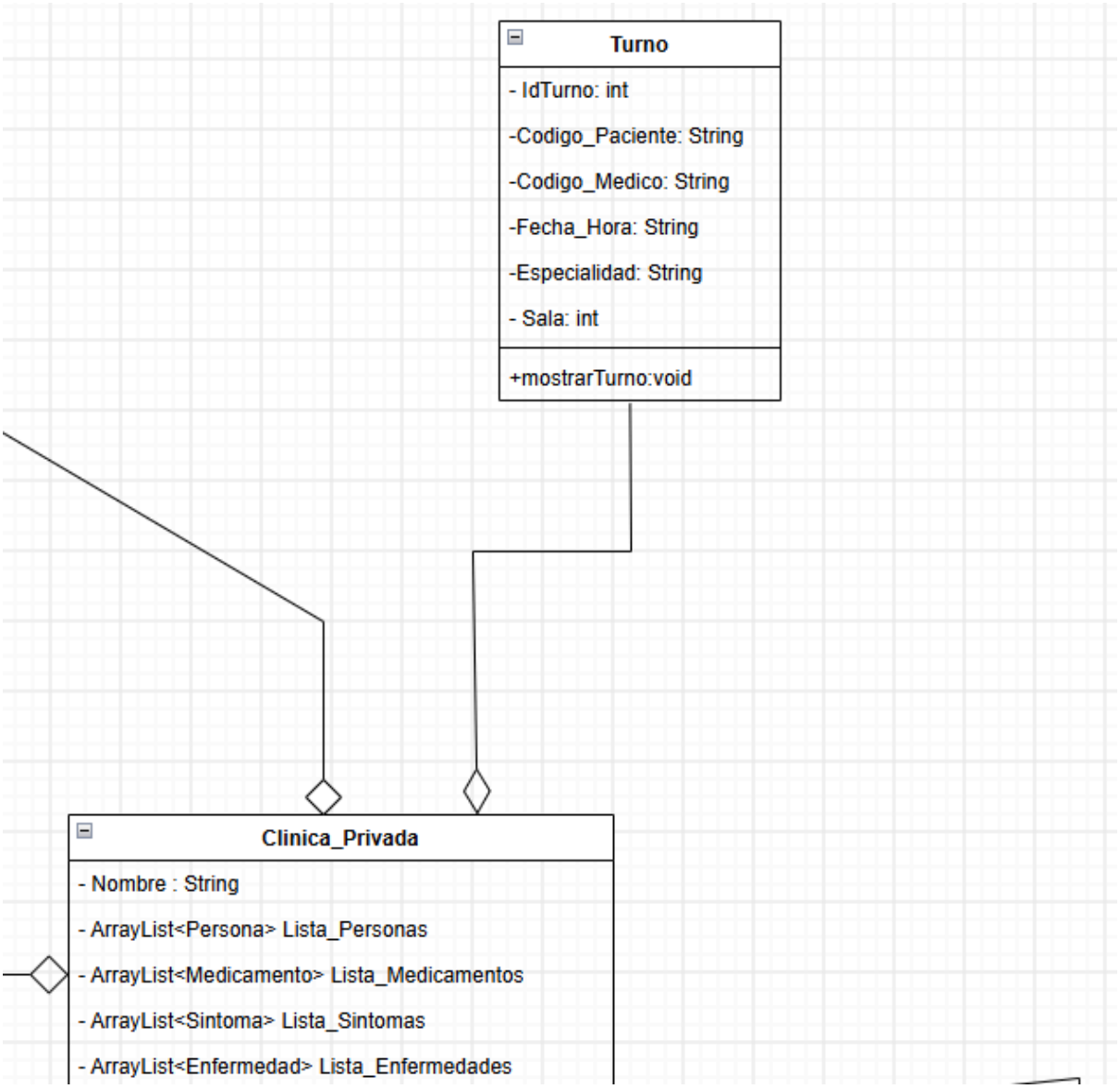
public class Turno {
    private int IdTurno;
    private String Codigo_Paciente;
    private String Codigo_Medico;
    private String Fecha_Hora;
    private String Especialidad ;
    private int Sala;

    public Turno(int IdTurno, String Codigo_Paciente, String Codigo_Medico, String Fecha_Hora, String Especialidad, int Sala) {
        this.IdTurno = IdTurno;
        this.Codigo_Paciente = Codigo_Paciente;
        this.Codigo_Medico = Codigo_Medico;
        this.Fecha_Hora = Fecha_Hora;
        this.Especialidad = Especialidad;
        this.Sala = Sala;
    }
```

Esto sería lo que printea el programa cuando se llama al método “mostrarTurno”.

```java
    public void mostrarTurno(){
        JOptionPane.showMessageDialog(null, "------- Turnos -------\n" +
                "Id Turno : " + IdTurno + "\n" +
                "Codigo del Paciente: " + Codigo_Paciente + "\n" +
                "Codigo del Medico: " + Codigo_Medico + "\n" +
                "Fecha Y Hora: " + Fecha_Hora+"\n"+
                "Especialidad: " + Especialidad+"\n"+
                "Sala: "+Sala);
    }
```

A continuación muestro la clase “Turno” conectada a “Clinica_Privada” en el UML, que se realizó con Matias Basto.



Luego con Matias se empezó a realizar la clase controladora Turnos. También se empezó con la clase controladora de la clase Medicamentos.

Esto sería parte de las funciones para la clase Turno, mostrando agregar y editar.

El método “AgregarTurno” añade el turno a la lista asignada.
El método “editarTurno” busca según el “id” del turno.

```
    //++++++++++++++++++++++++{ TURNO }++++++++++++++++++++++++++++++++++++//

    //agregar TURNO
    public void Agregar_Turno(Turno t) {
        Lista_Turno.add(t);
    }

    //editar TURNO


public void editarTurno(int IdTurno) {
    Turno tu = buscar_Turno(IdTurno);

    if (tu == null) {
        JOptionPane.showMessageDialog(null, "No se encontró ningún Turno con ese id.");
        return;
    }


    String Nuevo_Codigo_Paciente = JOptionPane.showInputDialog("Ingrese el nuevo código del paciente:");
    String Nuevo_Codigo_Medico = JOptionPane.showInputDialog("Ingrese el nuevo código del médico:");
    String Nuevo_Fecha_Hora = JOptionPane.showInputDialog("Ingrese la nueva fecha y hora del turno:");
    String Nuevo_Especialidad = JOptionPane.showInputDialog("Ingrese la nueva especialidad:");
    int Nueva_sala = Integer.parseInt(JOptionPane.showInputDialog("Ingrese la sala de consultorio:"));


    tu.setCodigo_Paciente(Nuevo_Codigo_Paciente);
    tu.setCodigo_Medico(Nuevo_Codigo_Medico);
```

```
    tu.setCodigo_Paciente(Nuevo_Codigo_Paciente);
    tu.setCodigo_Medico(Nuevo_Codigo_Medico);
    tu.setFecha_Hora(Nuevo_Fecha_Hora);
    tu.setEspecialidad(Nuevo_Especialidad);
    tu.setSala(Nueva_sala);
```

Y esto sería la función de “Eliminar_Turno” que funciona por el “ID”.
También está la función de “buscar_Turno”, que también es por “ID”.

```
   //ELIMINAR TURNO

   public void Eliminar_Turno(int IdTurno){
      Turno tu = buscar_Turno(IdTurno);

      if (tu !=null){
          Lista_Turno.remove(tu);
          JOptionPane.showMessageDialog(null, "Turno" + IdTurno + " Eliminado con exito.");
      }else{
          JOptionPane.showMessageDialog(null, "Turno con el ID " + IdTurno + " no encontrado.");
      }


   }

 //BUSCAR TURNO

  public Turno buscar_Turno( int IdTurno){
      for(Turno tu : Lista_Turno){
          if(tu.IdTurno == IdTurno){
              tu.mostrarTurno();
              return tu;
          }
      }

      return null;
  }
```

Y esto sería mostrar, que printea los datos que tenga la lista guardada.

```
//MOSTRAR TURNO

    public void mostrarTodosLosTurnos() {
    if (Lista_Turno.isEmpty()) {
        JOptionPane.showMessageDialog(null, "No hay turnos registrados.");
        return;
    }
    for (Turno tu : Lista_Turno) {
        tu.mostrarTurno();
    }
}
```

Esto fue lo que comencé de la clase controladora Medicamentos, solo le puse las funciones de agregar y buscar.

El método “AgregarMedicamento” añade el medicamento a la lista asignada.
El método “Buscar_medicamento” busca según el nombre del medicamento.

```
//++++++++++++++++++++++++{ Medicamento }++++++++++++++++++++++++++++++++++++++++++//
    //Agregar Medicamento
public void AgregarMedicamento (Medicamento m){
    Lista_Medicamentos.add(m);
}

    //Buscar Medicamento
public Medicamento Buscar_Medicamento (String Nombre) {
    for (Medicamento m : Lista_Medicamentos) {
        if (m.getNombre().equalsIgnoreCase(Nombre)) {
            return m;
        }
    }
    return null;
}
```

05/09
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

06/09
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

07/09
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

08/09
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

09/09
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

10/09
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

11/09
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

12/09
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

13/09
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

14/09
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

15/09
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

16/09
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

17/09
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

18/09
Termine la clase controladora de Medicamentos, agregándoles las funciones que me faltaban que eran, eliminar medicamento, y editar.

La función eliminar elimina según el nombre del medicamento que deseamos eliminar.

```
public void EliminarMedicamento(String Nombre){
    Medicamento m = Buscar_Medicamento(Nombre);
    if(m != null){
        Lista_Medicamentos.remove(m);
        JOptionPane.showMessageDialog(null, "Medicamento Eliminado.");
    }else{
        JOptionPane.showMessageDialog(null, "Medicamento no encontrado, intentelo de nuevo.");
    }
}
```

La función de editar funciona con el nombre del medicamento que deseamos editar.

```java
    //Editar Medicamento
public void Editar_Medicamento(String Nombre) {
    Medicamento m = Buscar_Medicamento(Nombre);

if (m == null) {
    JOptionPane.showMessageDialog(null, "No se encontró ningún medicamento con el nombre: " + Nombre);
    return;
}

String nuevoNombre = JOptionPane.showInputDialog("Ingrese el nuevo nombre del medicamento:");
String nuevoGrupo_Anatomico = JOptionPane.showInputDialog("Ingrese el nuevo grupo anatomico del medicamento:");
String nuevoTipo = JOptionPane.showInputDialog("Ingrese el nuevo tipo del medicamento:");
String nuevaCondicion_Venta = JOptionPane.showInputDialog("Ingrese la nueva condicion de venta del medicamento:");
String nuevaForma_Farmaceutica = JOptionPane.showInputDialog("Ingrese la nueva forma Farmaceutica del medicamento:");
String nuevaCategoria_Subtipo = JOptionPane.showInputDialog("Ingrese la nueva categoria Subtipo del medicamento:");
String nuevaVia_Administracion = JOptionPane.showInputDialog("Ingrese la nueva via de Administracion del medicamento:");
String nuevaDosis_Unidad = JOptionPane.showInputDialog("Ingrese la nueva dosis por Unidad del medicamento:");
int nuevaDosis_Cantidad = Integer.parseInt(JOptionPane.showInputDialog("Ingrese la nueva dosis por cantidad del mediamento : "));

m.setNombre(nuevoNombre);
m.setGrupo_Anatomico(nuevoGrupo_Anatomico);
m.setTipo(nuevoTipo);
m.setCondicion_Venta(nuevaCondicion_Venta);
m.setForma_Farmaceutica(nuevaForma_Farmaceutica);
m.setCategoria_Subtipo(nuevaCategoria_Subtipo);
m.setVia_Administracion(nuevaVia_Administracion);
m.setDosis_Unidad(nuevaDosis_Unidad);
m.setDosis_Cantidad(nuevaDosis_Cantidad);
```

```java
JOptionPane.showMessageDialog(null, "Medicamento actualizado correctamente");
}
```

También empecé a realizar la de Enfermedades, solo le agregue la función de agregar.

Que agrega la enfermedad en la lista.

```java
//+++++++++++++++++++++++++{ Emfermedad }++++++++++++++++++++++++++++++++++++++++//
    //Agregar Enfermedad
public void AgregarEnfermedad (Enfermedad e){
    Lista_Enfermedad.add(e);
}
```

19/09
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

20/09
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

21/09
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

22/09
Finalice la clase controladora Enfermedades y se la agrego al código principal, junto a la de medicamentos. A continuación adjunto imágenes y una breve explicación del código.

La función buscar actúa usando el “id” de la enfermedad que se desea buscar.

Y la función eliminar también usa el “id” de la enfermedad, solo que en este caso es para eliminar dicha enfermedad.

```
//Buscar
public Enfermedad Buscar_Enfermedad(int idEnfermedad){
    for(Enfermedad e: Lista_Enfermedad){
        if(e.getIdEnfermedad() == idEnfermedad){
            return e;
        }else{
            JOptionPane.showMessageDialog(null , "No hay ninguna Enfermedad asignada a ese ID.");
        }
    }
    return null;
}

//Eliminar
public void Eliminar_Enfermedad (int idEnfermedad){
    Enfermedad e = Buscar_Enfermedad(idEnfermedad);
    if(e != null){
        Lista_Enfermedad.remove(e);
        JOptionPane.showMessageDialog(null, "Enfermedad Eliminada.");
    }else{
        JOptionPane.showMessageDialog(null, "Enfermedad no encontrada, intentelo de nuevo.");
    }
}
```

Y la función editar también usa el “id” de la enfermedad, pero esta vez es para editar todos o algún dato en concreto de la clase “Enfermedad”.

```
//Editar
public void editarEnfermedad(int idEnfermedad) {
Enfermedad e = Buscar_Enfermedad(idEnfermedad);

if (e == null) {
    JOptionPane.showMessageDialog(null, "No se encontró ninguna Enfermedad  con ese ID.");
    return;
}
String nuevo_Nombre = JOptionPane.showInputDialog("Ingrese el nuevo nombre de la enfermead");
String nueva_Descripcion = JOptionPane.showInputDialog("Ingrese la nueva descripcion de la enfermead");
String nueva_Categoria = JOptionPane.showInputDialog("Ingrese la nueva categoria de la enfermead");
String nueva_Lista_Sintomas = JOptionPane.showInputDialog("Ingrese la nueva Lista de Sintomas de la enfermead");
String nuevo_Lista_Tratamientos = JOptionPane.showInputDialog("Ingrese la nueva Lista de Tratamientos de la enfermead");
String nueva_Observacion = JOptionPane.showInputDialog("Ingrese la nueva observacion de la enfermead");

e.setNombre(nuevo_Nombre);
e.setDescripcion(nueva_Descripcion);
e.setCategoria(nueva_Categoria);
e.setLista_Sintomas(nueva_Lista_Sintomas);
e.setLista_Tratamientos(nuevo_Lista_Tratamientos);
e.setObservaciones(nueva_Observacion);

JOptionPane.showMessageDialog(null, "Enfermedad actualizada correctamente");
}
```

23/09
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

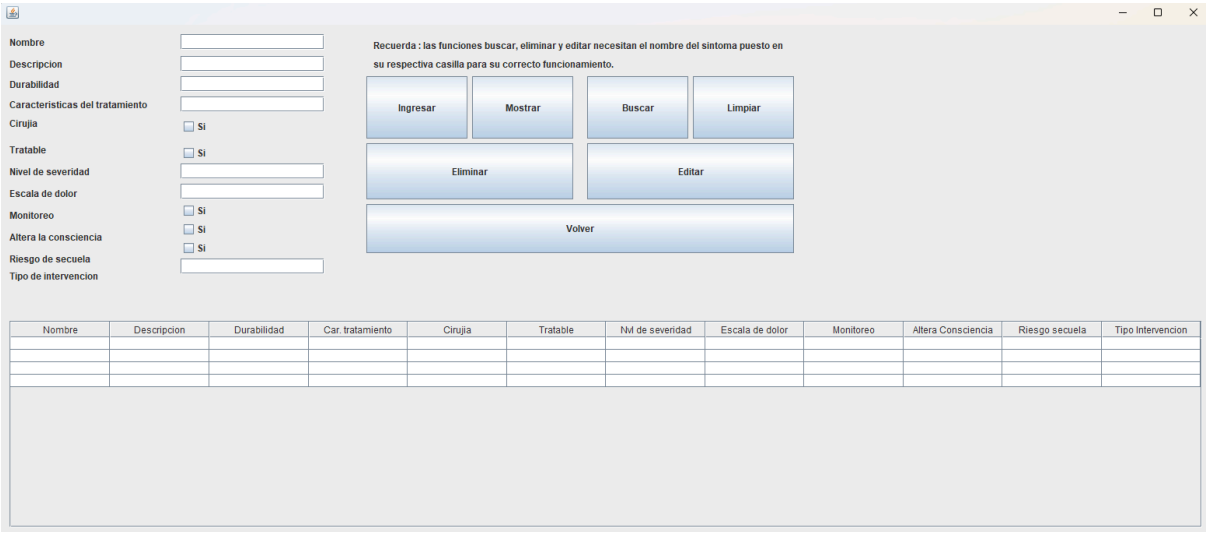
24/09
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

25/09
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

26/09
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

27/09
Este dia me puse a realizar el JFrame de la clase Síntoma agregando sus respectivas funciones y agregue unos botones para las opciones, ya que son booleanos.

El botón “JCheckBox” funciona de la siguiente manera, si no se lo selecciona va quedar con el valor “False” y si se clickea va a pasar a “True”.



Esta es la parte del código de la función “Ingresar”.

```
private void btnIngresarActionPerformed(java.awt.event.ActionEvent evt) {
    String Nombre = txtNombre.getText();
    String Descripcion = txtDescripcion.getText();
    String Durabilidad = txtDurabilidad.getText();
    String Caracteristicas_Tratamiento = txtCTratamiento.getText();
    boolean Cirujia = chkCirujia.isSelected();
    boolean Tratable = chkTratable.isSelected();
    String Nivel_Severidad = txtNvlSeveridad.getText();
    int Escala_Dolor = Integer.parseInt(txtEscDolor.getText());
    boolean Requiere_Monitoreo = chkMonitoreo.isSelected();
    boolean Altera_Consciencia = chkAltConsciencia.isSelected();
    boolean Riesgo_Secuela = chkRiesgoSecuela.isSelected();
    String Tipo_Intervencion = txtTipoIntervencion.getText();

    Sintoma nuevoSintoma = new Sintoma(Nombre,Descripcion,Durabilidad,Caracteristicas_Tratamiento,Cirujia,Tratable,Nivel_Severidad,Es
    CP.Agregar_Sintoma(nuevoSintoma);

    JOptionPane.showMessageDialog(null, "Sintoma registrado exitosamente !");
    txtNombre.setText("");
    txtDescripcion.setText("");
    txtDurabilidad.setText("");
    txtCTratamiento.setText("");
    chkCirujia.setSelected(false);
    chkTratable.setSelected(false);
    txtNvlSeveridad.setText("");
    txtEscDolor.setText("");
    chkMonitoreo.setSelected(false);
    chkAltConsciencia.setSelected(false);
    chkRiesgoSecuela.setSelected(false);
```

Esto pertenece a la parte de buscar según el nombre del síntoma.

```
private void btnBuscarActionPerformed(java.awt.event.ActionEvent evt) {
    String nombreBuscado = txtNombre.getText();
    Sintoma s = CP.Buscar_Sintoma(nombreBuscado);

    if(s instanceof Sintoma && s!=null){
        Sintoma Sin =(Sintoma)s;
        mostrarSintomaEnTabla(Sin);
    }
    else{
        JOptionPane.showMessageDialog(null , "No hay ningun sintoma con ese nombre.");
    }
```

Y estas son funciones mostrar, editar y limpiar.

```
private void btnMostrarActionPerformed(java.awt.event.ActionEvent evt) {
    actualizarTabla();
}

private void btnEditarActionPerformed(java.awt.event.ActionEvent evt) {
    CP.Editar_Sintoma(txtNombre.getText());
}

private void btnLimpiarActionPerformed(java.awt.event.ActionEvent evt) {
    txtNombre.setText("");
    txtDescripcion.setText("");
    txtDurabilidad.setText("");
    txtCTratamiento.setText("");
    chkCirujia.setSelected(false);
    chkTratable.setSelected(false);
    txtNvlSeveridad.setText("");
    txtEscDolor.setText("");
    chkMonitoreo.setSelected(false);
    chkAltConsciencia.setSelected(false);
    chkRiesgoSecuela.setSelected(false);
    txtTipoIntervencion.setText("");
```

28/09
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

29/09
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

30/09
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

01/10
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

02/10
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

03/10
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

04/10
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

05/10
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

06/10
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

07/10
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

08/10
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

09/10
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

10/10
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

11/10
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

12/10
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

13/10
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

14/10
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

15/10
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

16/10
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

17/10

Este día no se realizó ninguna tarea/actividad sobre el proyecto.

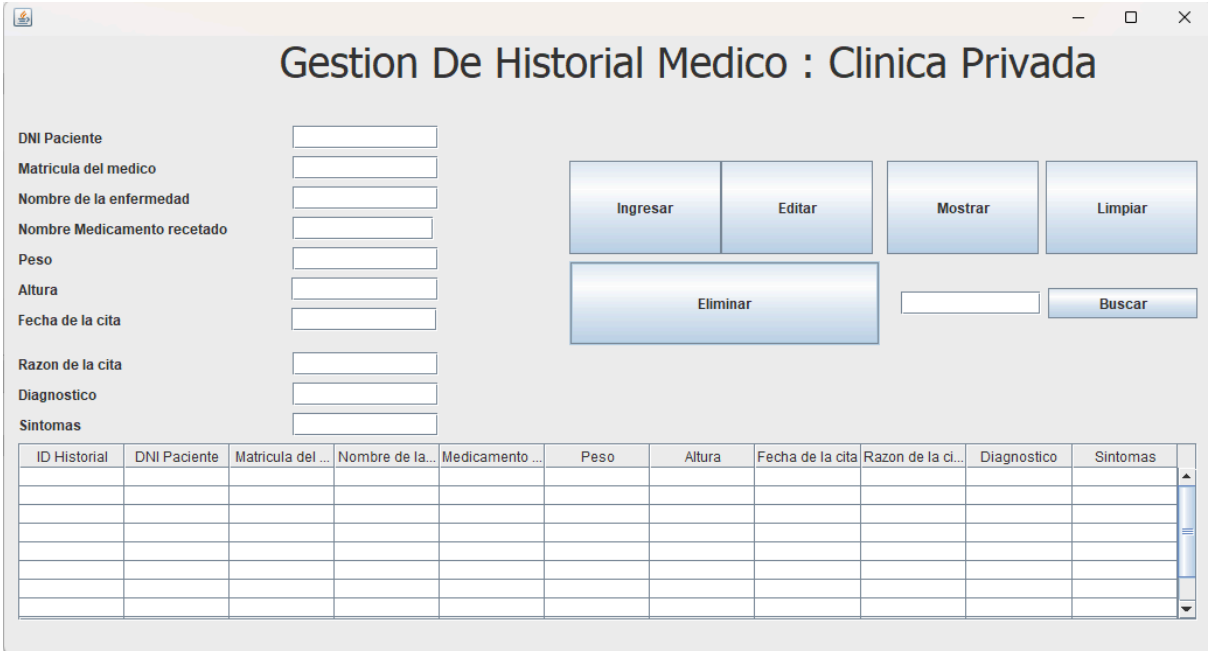
18/10
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

19/10
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

20/10
Después de 3 intentos de tratar de arreglar el porqué no funciona el JFrame de Historial_Medico,le consulte a Matías Basto y él le preguntó al profesor cuál era el problema, y le dijo que era un problema de los botones. El profesor había dicho que el problema se debía a que se habrá hecho algún evento de más y por eso tiraba error, para solucionarlo se tenían que eliminar todos los botones y volver a programarlos uno por uno. Pero seguía tirando error, entonces decidí eliminar todo el JFrame y volver a hacerlo, de esta manera conseguí que no me diera errores.

También se tuvo que modificar la base de datos porque no tenían datos que sí tenía el JFrame, además se le agregaron un par de cosas más.

Esto es lo que vería el Médico al iniciar el JFrame de Historial Médico.



Esta es la función del botón “Ingresar”.

```
private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {
    int DNIP = Integer.parseInt(txtDNIP.getText());
    String MatriculaMed = txtMatricula.getText();
    String NombEnfermedad = txtEnfermedad.getText();
    String NombMedicamento = txtMedicamento.getText();
    int Peso = Integer.parseInt(txtPeso.getText());
    int Altura = Integer.parseInt(txtAltura.getText());
    String FechaCita = txtFecha.getText();
    String RazonCita = txtRazon.getText();
    String Diagnostico = txtDiagnostico.getText();
    String Sintomas = txtSintomas.getText();

    Historial_Medico HistMed = new Historial_Medico(DNIP,MatriculaMed,NombEnfermedad,NombMedicamento,FechaCita,Peso,Altura,RazonCita,I
    Historial_MedicoDAO dao = new Historial_MedicoDAO();
    dao.insertar(HistMed);

    JOptionPane.showMessageDialog(null, "Historial Medico registrado exitosamente !");

    txtDNIP.setText("");
    txtMatricula.setText("");
    txtEnfermedad.setText("");
    txtMedicamento.setText("");
    txtPeso.setText("");
    txtAltura.setText("");
    txtFecha.setText("");
    txtRazon.setText("");
    txtDiagnostico.setText("");
    txtSintomas.setText("");
}
```

Esta es la funcion del boton “Mostrar”

```
private void btnMostrarActionPerformed(java.awt.event.ActionEvent evt) {
    Historial_MedicoDAO dao = new Historial_MedicoDAO();
    ArrayList<Historial_Medico> lista_Historial = dao.listar();
    DefaultTableModel modelo = (DefaultTableModel) tablaHistMedico.getModel();

    modelo.setRowCount(0);

    for (Historial_Medico HistMed : lista_Historial) {
        modelo.addRow(new Object[]{
            HistMed.getId_historial(),
            HistMed.getDNI_Paciente(),
            HistMed.getId_medico(),
            HistMed.getNombreDeLaEnfermedad(),
            HistMed.getNombreMedicamento(),
            HistMed.getPeso(),
            HistMed.getAltura(),
            HistMed.getFecha_Cita(),
            HistMed.getRazon_Cita(),
            HistMed.getDiagnostico(),
            HistMed.getLista_Sintomas(),

        });
    }
}
```

Esta es la función del botón “Eliminar”, que para que funcione hay que hacer click sobre alguna fila para seleccionar el registro que se desea eliminar.

```
private void btnEliminarActionPerformed(java.awt.event.ActionEvent evt) {
    int fila = tablaHistMedico.getSelectedRow();

    if (fila == -1) {
        JOptionPane.showMessageDialog(null, "Debe seleccionar un historial");
        return;
    }

    int idHistorial = Integer.parseInt(tablaHistMedico.getValueAt(fila, 0).toString());

    Historial_MedicoDAO dao = new Historial_MedicoDAO();
    dao.eliminar(idHistorial);

    JOptionPane.showMessageDialog(null, "Registro eliminado exitosamente");
}
```

Esta es la función del botón “Buscar”, que funciona ingresando en el JTextField el DNI del paciente que se desea buscar.

```
private void btnBuscarActionPerformed(java.awt.event.ActionEvent evt) {
    String DNIP_Buscado = txtDNI_Buscado.getText();

Historial_MedicoDAO dao = new Historial_MedicoDAO();

ArrayList<Historial_Medico> lista_HistMed = dao.buscarPorDNIPaciente(DNIP_Buscado);

DefaultTableModel modelo = (DefaultTableModel) tablaHistMedico.getModel();

modelo.setRowCount(0);

for (Historial_Medico HistMed : lista_HistMed) {
    modelo.addRow(new Object[]{
        HistMed.getId_historial(),
            HistMed.getDNI_Paciente(),
            HistMed.getId_medico(),
            HistMed.getNombreDeLaEnfermedad(),
            HistMed.getNombreMedicamento(),
            HistMed.getPeso(),
            HistMed.getAltura(),
            HistMed.getFecha_Cita(),
            HistMed.getRazon_Cita(),
            HistMed.getDiagnostico(),
            HistMed.getLista_Sintomas(),
    });
}
}
```

Esta es la función del botón “Editar”, que para que funcione hay que hacer click sobre alguna fila para seleccionar el registro que se desea editar, como en eliminar.

```
private void btnEditarActionPerformed(java.awt.event.ActionEvent evt) {
    int fila = tablaHistMedico.getSelectedRow();
    if (fila == -1) {
        System.out.println("Debe seleccionar un historial.");
        return;
    }
    int id = (int) tablaHistMedico.getValueAt(fila, 0);
    int DNIP = Integer.parseInt(txtDNIP.getText());
    String MatriculaMed = txtMatricula.getText();
    String NombEnfermedad = txtEnfermedad.getText();
    String NombMedicamento = txtMedicamento.getText();
    int Peso = Integer.parseInt(txtPeso.getText());
    int Altura = Integer.parseInt(txtAltura.getText());
    String FechaCita = txtFecha.getText();
    String RazonCita = txtRazon.getText();
    String Diagnostico = txtDiagnostico.getText();
    String Sintomas = txtSintomas.getText();

    Historial_Medico HistMed = new Historial_Medico();
    HistMed.setId_historial(id);
    HistMed.setDNI_Paciente(DNIP);
    HistMed.setId_medico(MatriculaMed);
    HistMed.setNombreDeLaEnfermedad(NombEnfermedad);
    HistMed.setNombreMedicamento(NombMedicamento);
    HistMed.setPeso(Peso);
    HistMed.setAltura(Altura);
    HistMed.setFecha_Cita(FechaCita);
    HistMed.setRazon_Cita(RazonCita);
    HistMed.setDiagnostico(Diagnostico);


    HistMed.setDiagnostico(Diagnostico);
    HistMed.setLista_Sintomas(Sintomas);

    Historial_MedicoDAO dao = new Historial_MedicoDAO();
    dao.actualizar(HistMed);
    JOptionPane.showMessageDialog(null, "Datos modificados correctamente.");

}
```

Esta es la función del botón “Limpiar”.

```java
private void btnLimpiarActionPerformed(java.awt.event.ActionEvent evt) {
    txtDNIP.setText("");
    txtMatricula.setText("");
    txtEnfermedad.setText("");
    txtMedicamento.setText("");
    txtPeso.setText("");
    txtAltura.setText("");
    txtFecha.setText("");
    txtRazon.setText("");
    txtDiagnostico.setText("");
    txtSintomas.setText("");
}
```

Estos serían los cambios que se le hicieron a la base de datos (tablas).
Estos son los datos de la tabla “historial_medico” que estaban antes.

```sql
create table historial_medico (
    id_historial int primary key auto_increment,
    DNI_Paciente int not null,
    id_medico int not null,
    id_enfermedad int,
    id_medicamento int,
    fecha_cita varchar(100),
    peso int,
    altura int,
    razon_cita varchar(100),
    diagnostico varchar(100),
    lista_sintomas varchar(100),
    foreign key (id_medico) references medicos(id_medico),
    foreign key (id_enfermedad) references enfermedades(id_enfermedad),
    foreign key (id_medicamento) references medicamentos(id_medicamento)
);
```

Y estos son los datos de la tabla “historial_medico” que están ahora.

```sql
create table historial_medico (
    id_historial int primary key auto_increment,
    DNI_Paciente int not null,
    id_medico varchar(30),
    nombreDeLaEnfermedad varchar(100),
    nombreMedicamento varchar(100),
    fecha_cita varchar(100),
    peso int,
    altura int,
    razon_cita varchar(100),
    diagnostico varchar(100),
    lista_sintomas varchar(100),
    foreign key (id_medico) references medicos(Numero_Matricula),
    foreign key (nombreDeLaEnfermedad) references enfermedades(nombre),
    foreign key (nombreMedicamento) references medicamentos(nombre)
);
```

En la tabla medicamentos Matias le agregó el atributo “unique” a nombre, para que en la tabla “historial_medico” tomará el valor.

```sql
create table medicamentos (
    id_medicamento int auto_increment primary key ,
    nombre varchar(100),
    grupo_anatomico varchar(100),
    tipo varchar(50),
    condicion_venta varchar(50),
    forma_farmaceutica varchar(50),
    Categoria_Subtipo varchar(50),
    Via_Administracion varchar(50),
    Dosis_Unidad varchar(50),
    dosis_cantidad int
);
```

Acá se muestra el cambio del valor.

```sql
create table medicamentos (
    id_medicamento int auto_increment primary key ,
    nombre varchar(100) unique,
    grupo_anatomico varchar(100),
    tipo varchar(50),
    condicion_venta varchar(50),
    forma_farmaceutica varchar(50),
    Categoria_Subtipo varchar(50),
    Via_Administracion varchar(50),
    Dosis_Unidad varchar(50),
    dosis_cantidad int
);
```

Y los insert se tuvieron que modificar debido al cambio que se le hizo en la tabla “historial_medico”.

Estos eran los inserts que tenía antes la tabla “historial_medico”.

```sql
INSERT INTO historial_medico
(id_historial, DNI_Paciente, id_medico, id_enfermedad, id_medicamento, fecha_cita, peso, altura, razon_cita, diagnostico, lista_sintomas)
VALUES
(1, 40111222, 1, 1, 1, '2026-10-05 09:00:00', 75, 175, 'Control de rutina', 'Presion arterial elevada', 'Dolor de cabeza intenso, mareos, vision borrosa'),
(2, 40222333, 2, 2, 1, '2026-10-06 10:30:00', 60, 165, 'Fiebre y tos', 'Cuadro gripal', 'Fiebre alta, tos'),
(3, 40333444, 3, 3, 5, '2026-10-07 11:00:00', 82, 178, 'Dolor estomacal', 'Gastritis', 'Dolor estomacal, nauseas'),
(4, 40444555, 5, 4, 4, '2026-10-08 14:00:00', 68, 170, 'Picazon en la piel', 'Dermatitis', 'Picor, enrojecimiento'),
(5, 40555666, 4, 5, 3, '2026-10-09 15:30:00', 80, 180, 'Dolor de espalda', 'Lumbalgia', 'Dolor agudo, inflamacion');
```

Y estos son los inserts que tienen ahora.

```sql
INSERT INTO historial_medico (id_historial, DNI_Paciente, id_medico, nombreDeLaEnfermedad, nombreMedicamento, fecha_cita, peso, altura, razon_cita, diagnostico, lista_sintomas) VALUES
(1, 38452114, 'MN-10234', 'Hipertension arterial', 'Paracetamol', '2026-10-05 09:00:00', 75, 175, 'Control de rutina', 'Presion arterial elevada', 'Dolor de cabeza intenso, mareos, vision borrosa'),
(2, 41223658, 'MN-20456', 'Gripe', 'Amoxicilina', '2026-10-06 10:30:00', 60, 165, 'Fiebre and tos', 'Cuadro gripal', 'Fiebre alta, tos'),
(3, 24115887, 'MN-30567', 'Gastritis', 'Ibuprofeno', '2026-10-07 11:00:00', 82, 178, 'Dolor estomacal', 'Gastritis', 'Dolor estomacal, nauseas'),
(4, 45889123, 'MN-40678', 'Dermatitis', 'Loratadina', '2026-10-08 14:00:00', 68, 170, 'Picazon en la piel', 'Dermatitis', 'Picor, enrojecimiento'),
(5, 17458963, 'MN-50789', 'Lumbalgia', 'Omeprazol', '2026-10-09 15:30:00', 80, 180, 'Dolor de espalda', 'Lumbalgia', 'Dolor agudo, inflamacion');
```

También le agregue la función de buscar en “SintomaDAO”, ya que en ese momento que vi el archivo no la tenía.

```java
    public ArrayList<Sintoma> buscarSintoma(String Nombre) {
        ArrayList<Sintoma> sintomas = new ArrayList<>();
        String consulta_sql = "SELECT * FROM Sintoma WHERE Nombre = ?";

        try {
            Connection conexion = Conexion.conectar();
            PreparedStatement sentencia_sql = conexion.prepareStatement(consulta_sql);
            sentencia_sql.setString(1, Nombre);
            ResultSet resultado_sql = sentencia_sql.executeQuery();

            while (resultado_sql.next()) {
                Sintoma sin = new Sintoma();
                sin.setId_sintoma(resultado_sql.getInt("Id_sintoma"));
                sin.setNombre(resultado_sql.getString("Nombre"));
                sin.setDescripcion(resultado_sql.getString("Descripcion"));
                sin.setDurabilidad(resultado_sql.getString("Durabilidad"));
                sin.setCaracteristicas_Tratamiento(resultado_sql.getString("Caracteristicas_Tratamiento"));
                sin.setCirujia(resultado_sql.getString("Cirujia"));
                sin.setTratable(resultado_sql.getString("Tratable"));
                sin.setNivel_Severidad(resultado_sql.getString("Nivel_Severidad"));
                sin.setEscala_Dolor(resultado_sql.getInt("Escala_Dolor"));
                sin.setRequiere_Monitoreo(resultado_sql.getString("Requiere_Monitoreo"));
                sin.setAltera_Consciencia(resultado_sql.getString("Altera_Consciencia"));
                sin.setRiesgo_Secuela(resultado_sql.getString("Riesgo_Secuela"));
                sin.setTipo_Intervencion(resultado_sql.getString("Tipo_Intervencion"));

                sintomas.add(sin);
            }
        } catch (SQLException ex) {
            System.out.println("ERROR: " + ex.getMessage());
        }
        return sintomas;
    }
}
```

Por último vi que en el JFrame “VentanaBienvenida” estaba mal escrito el botón de “Iniciar Sesión”.
Así se veía antes.



Y así se ve ahora.



21/10
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

22/10
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

23/10
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

24/10
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

25/10
Hoy estuve en llamada con nuestro grupo realizando la maqueta de la clínica. La maqueta se realizó desde minecraft. Se me asignó hacer el cuarto piso de la clínica que era “Atención especializada”. A continuación comparto las imágenes de como fue quedando la clínica.

Estos son dos consultorios pequeños de psicología:

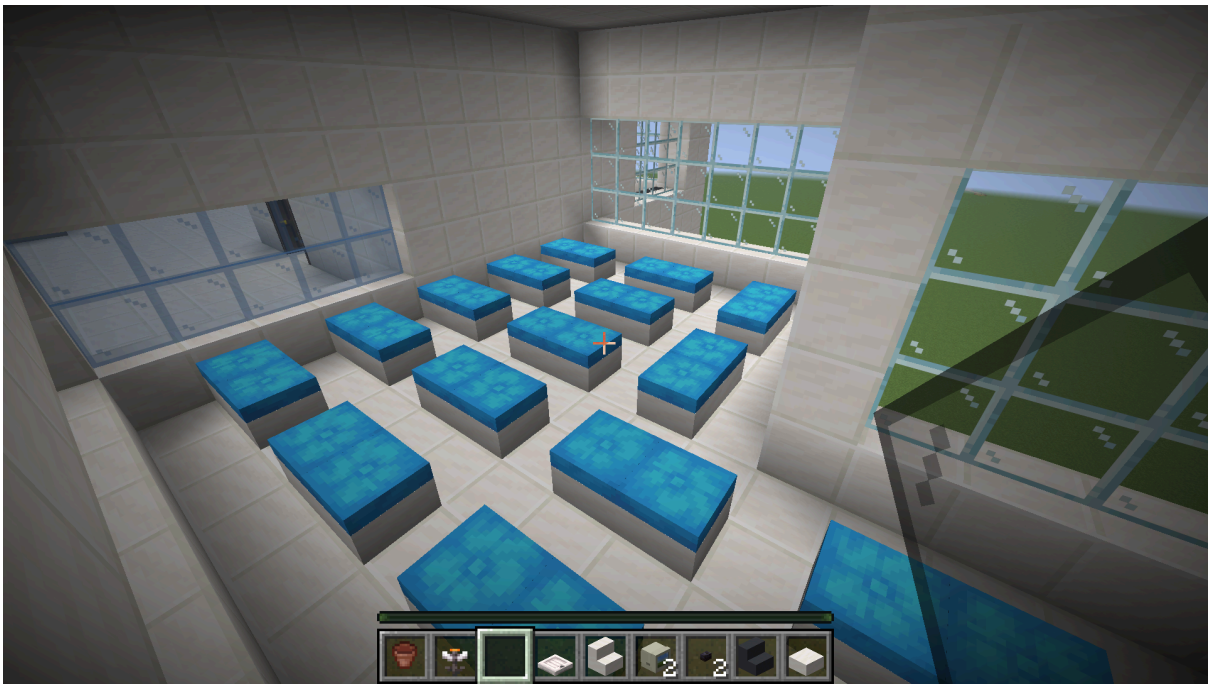




Esta es la zona de fisioterapia y rehabilitación.



Esto es maternidad, donde van los recién nacidos.

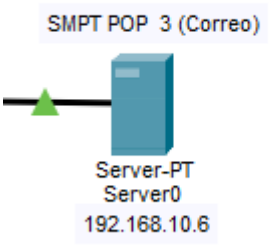


Y las siguientes dos imágenes serían la sala de conferencia de la clínica.

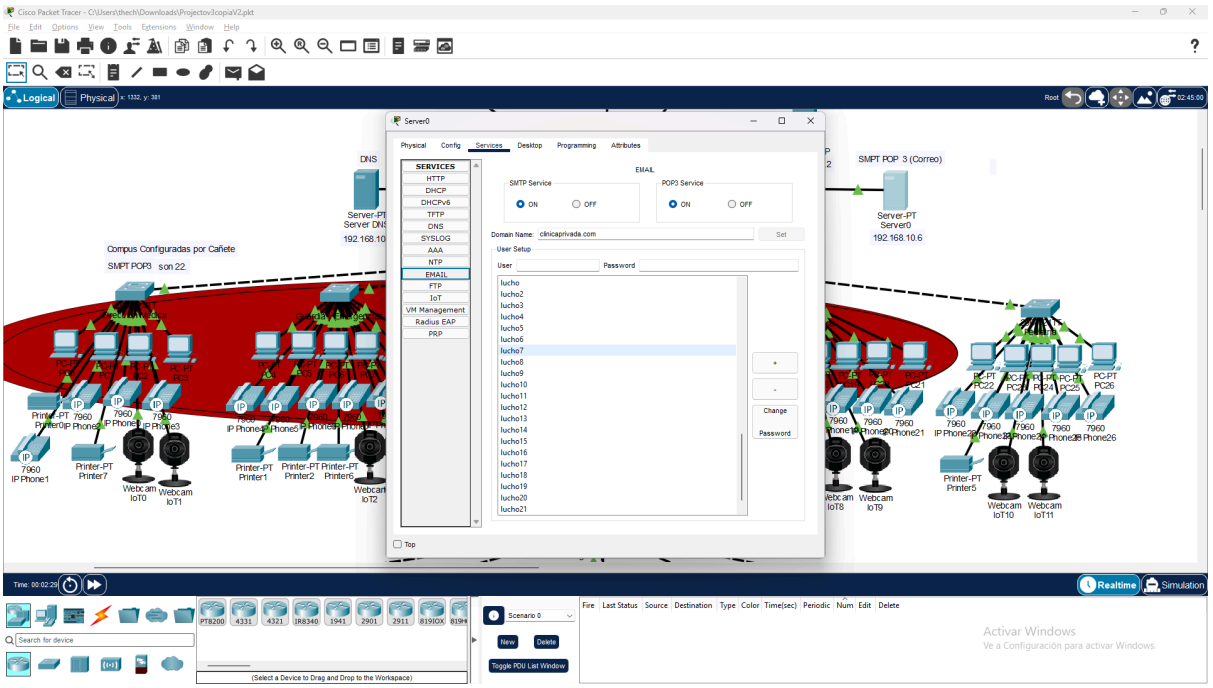


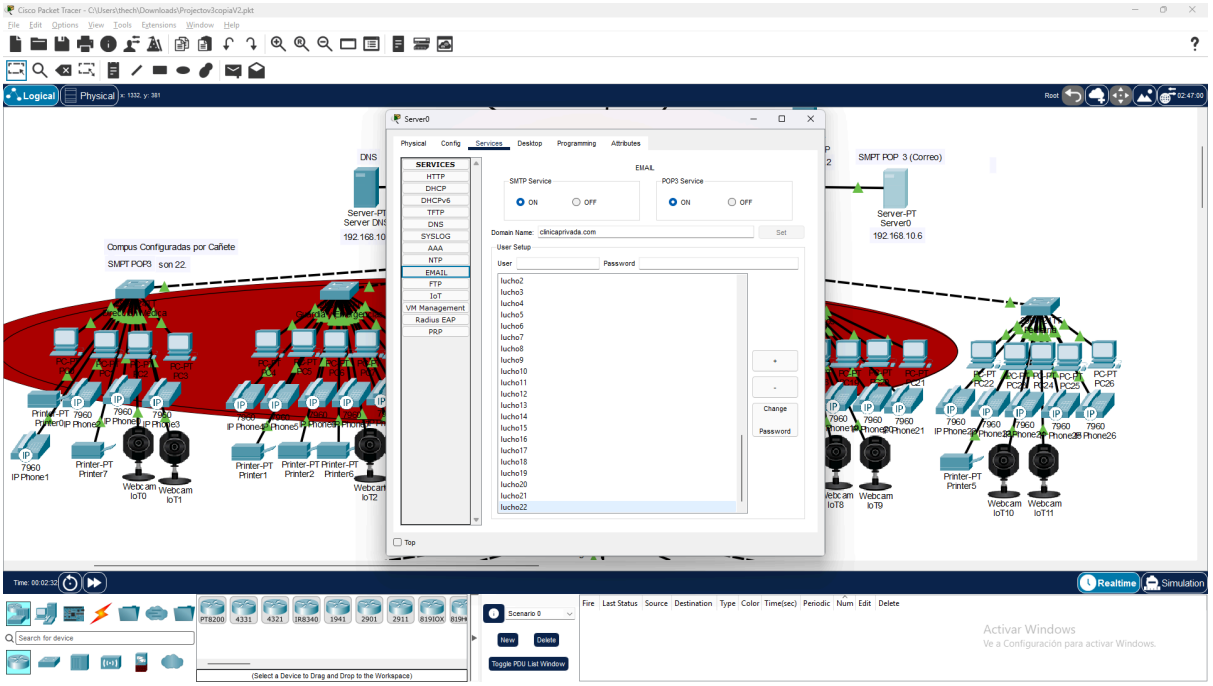


26/10
Hoy en la parte de redes, me puse a configurar los usuarios de 22 computadoras, asignándoles su respectivo nombre de usuario y la contraseña en el servidor de correo que tiene la ip 192.168.10.6

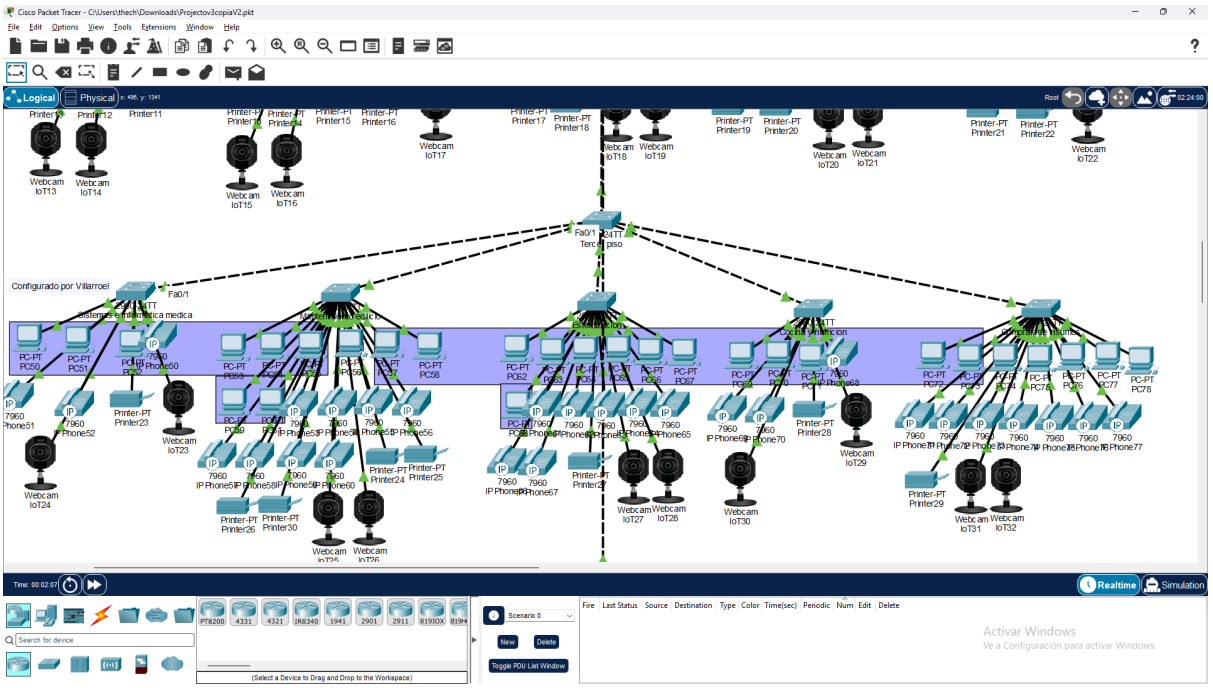


Después le configure a cada una de las computadoras su respectivo correo poniendo la información correspondiente según el nombre de usuario y contraseña que asigne en el servidor de correo.
Estos son los 22 usuarios configurados:

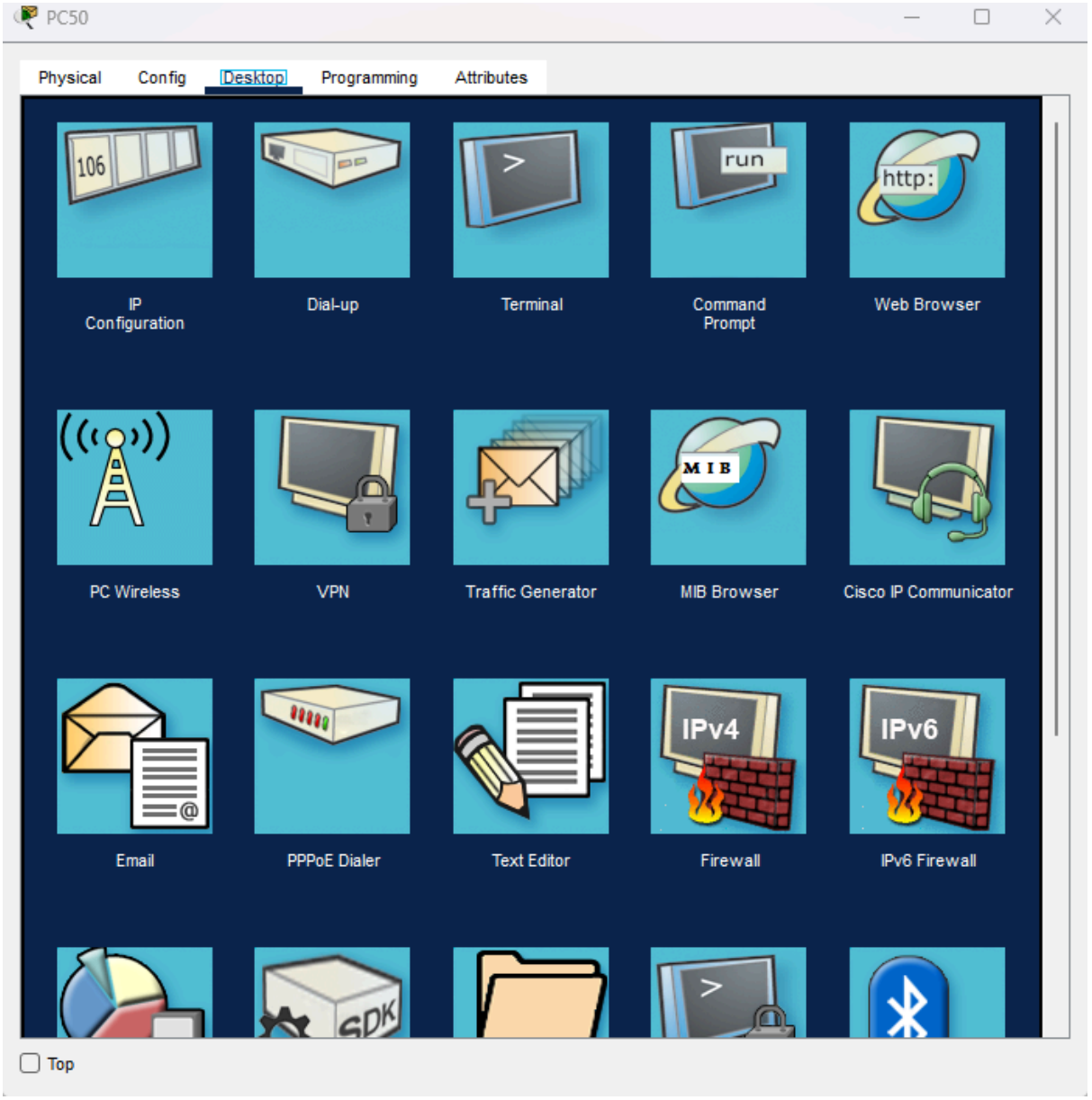


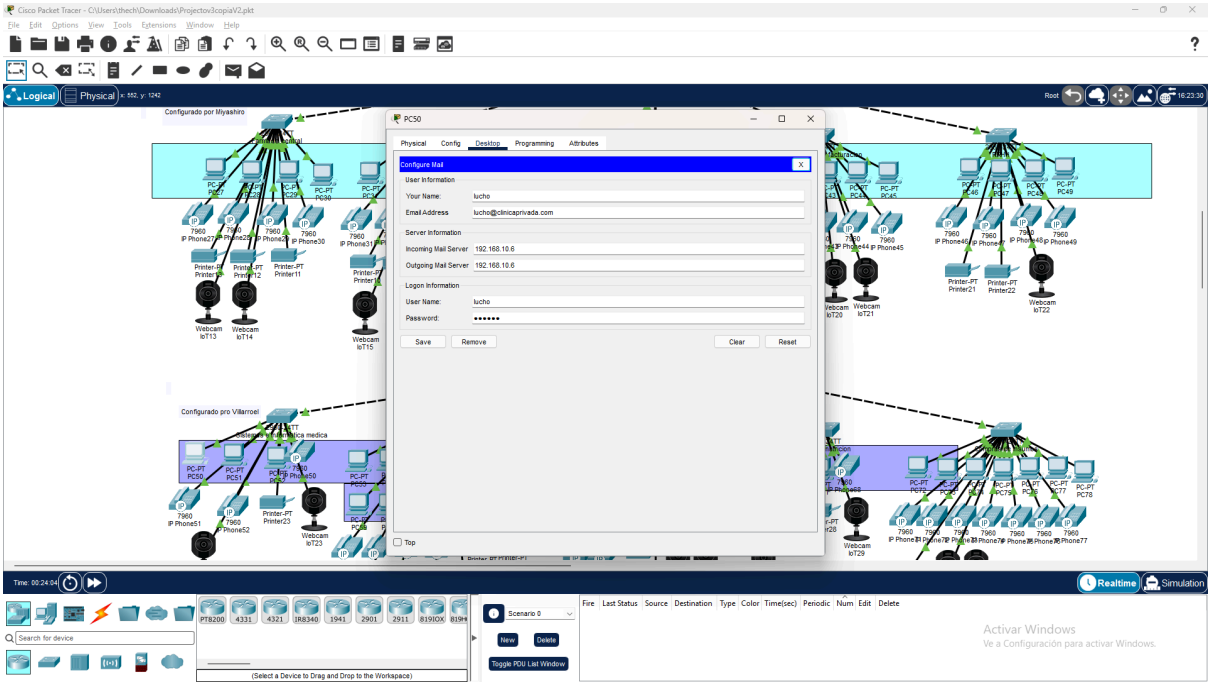


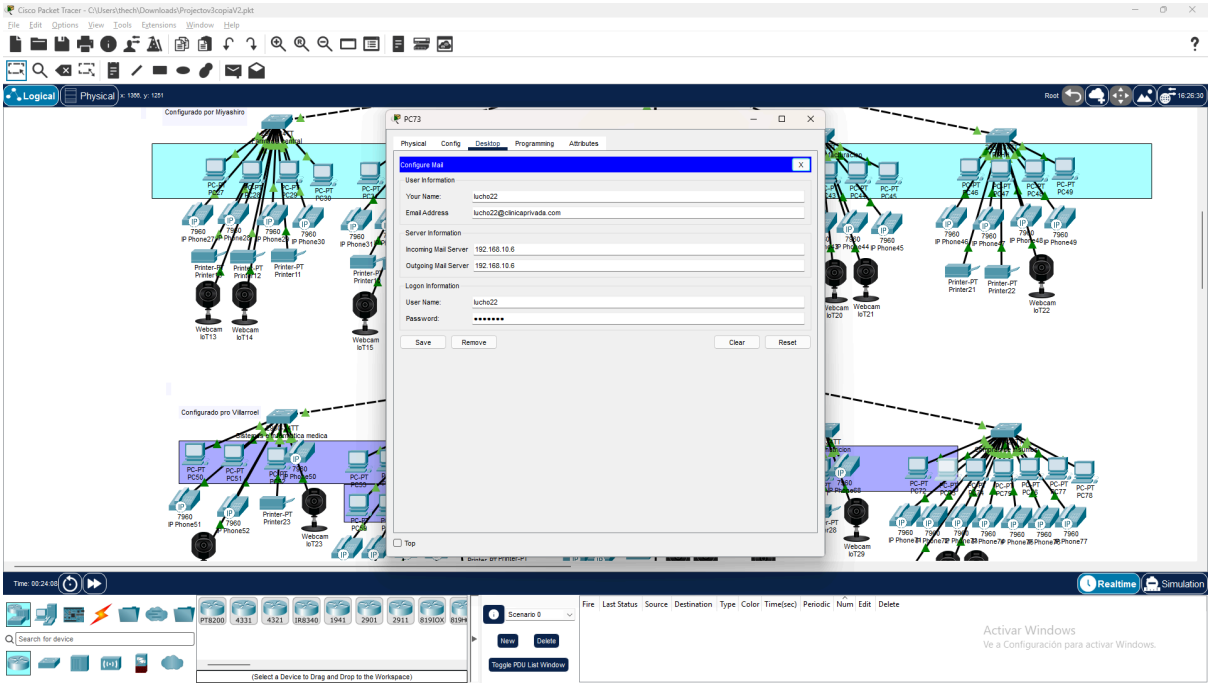
La franja violeta indica que computadoras configure.



Después en la parte de email dentro de las computadoras, les configure su respectiva información.


PC50
Physical
Config
Desktop
Programming
Attributes
106
IP Configuration
Dial-up
Terminal
run
Command Prompt
http:
Web Browser
PC Wireless
VPN
Traffic Generator
MIB
MIB Browser
Cisco IP Communicator
Email
PPPoE Dialer
Text Editor
IPv4
Firewall
IPv6
IPv6 Firewall
SDK
Top



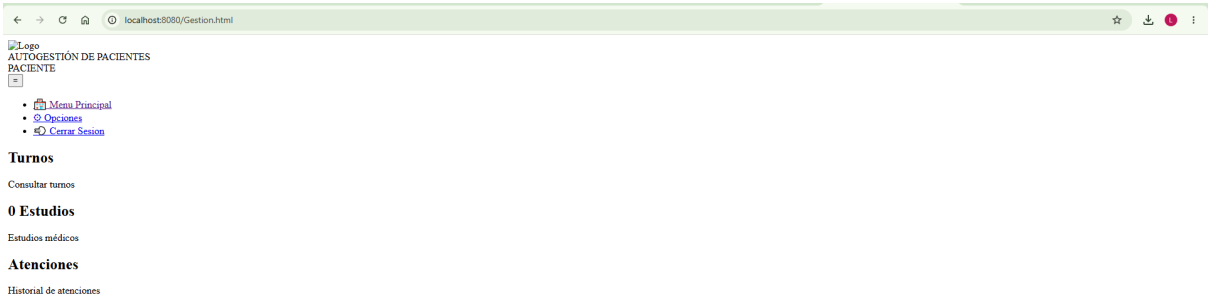


27/10

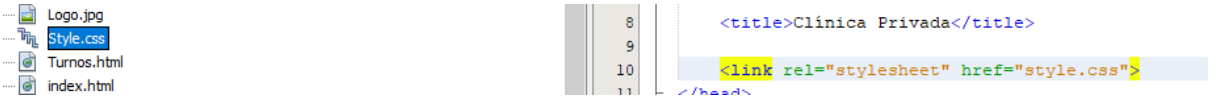
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

28/10

Luego de la revisión de Octubre se asignaron las nuevas tareas para la entrega final, que sería el 6 de noviembre. Se me asignó la tarea “Conexión Java - Web - SQL. Al descargar los archivos del GitHub y luego ejecutarlos. Me dio la siguiente url: http://localhost:8080 . Si se entra en la parte de Gestión va a mostrar lo siguiente:

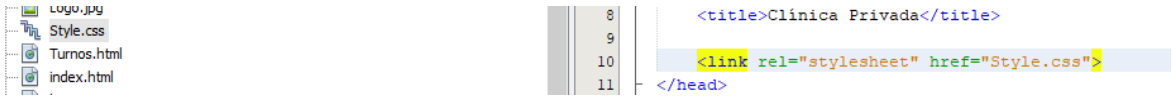


Esto está mal porque debería mostrar el css que tenemos. Al revisar el código de java encontré que estaba mal escrito el nombre del archivo, es solo cambiar el nombre.



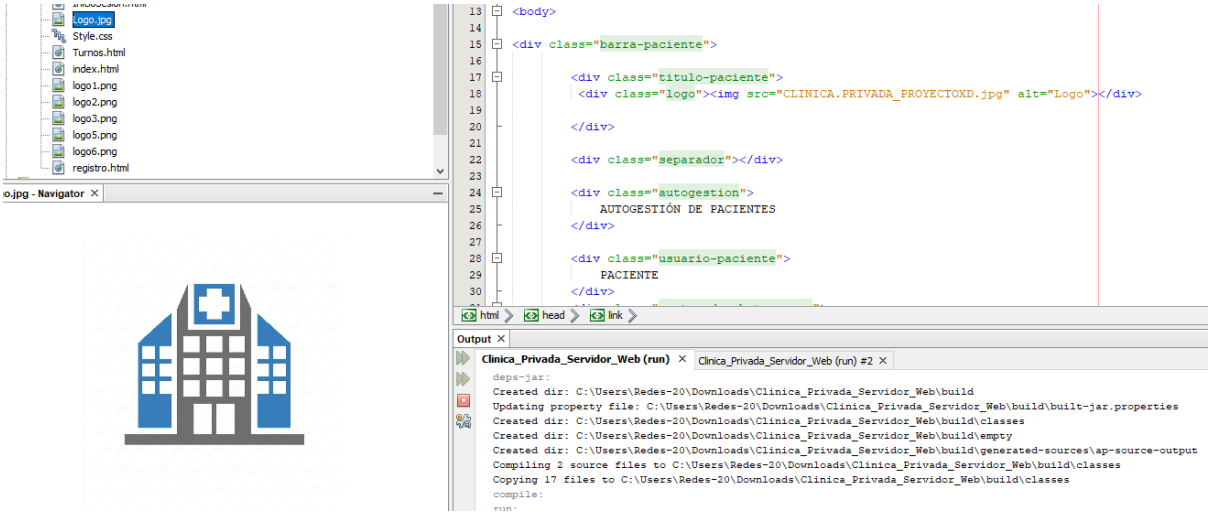
Quedando de la siguiente manera la página y el código:





Era cambiar en el código “style.css” por “Style.css”, que es el nombre del archivo css.

Viendo el código, vi que no mostraba las imágenes, como se había mostrado antes, aunque estuviera bien escrito el nombre de dicha imagen.





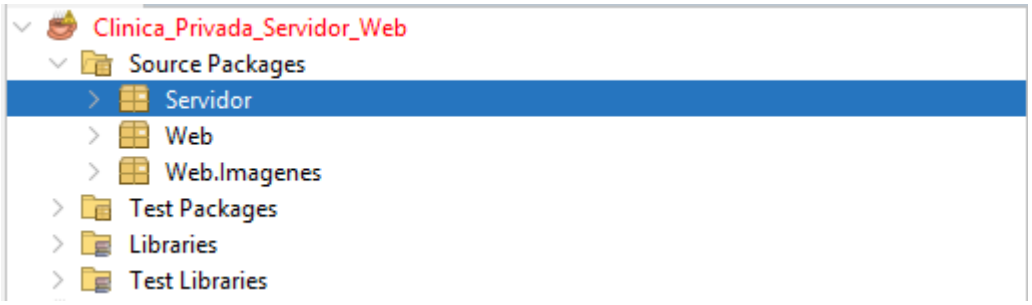
Estos avances no se guardaron, debido a que Santiago Ugaz estaba modificando desde su computadora el java para normalizar los botones, que es lo que nos aclararon en la entrega, luego lo modificaré de vuelta.

29/10
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

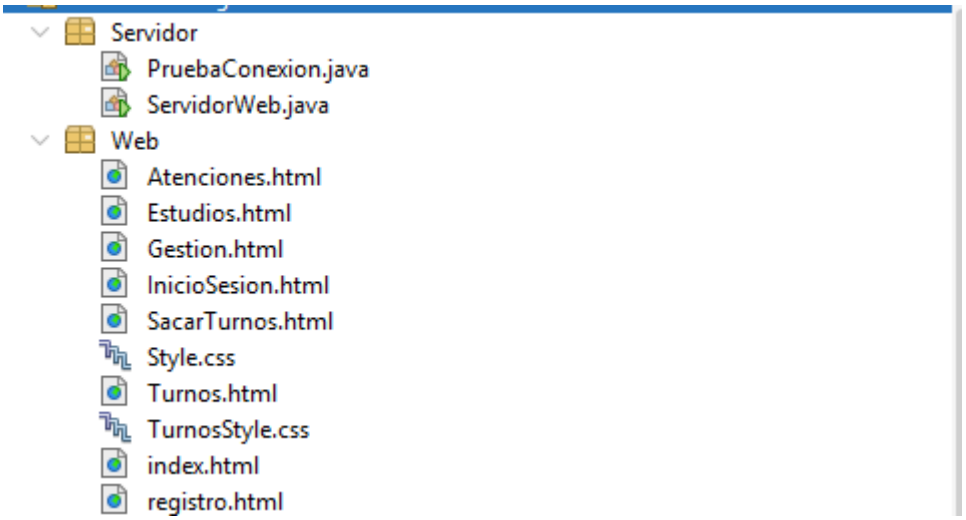
30/10
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

31/10
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

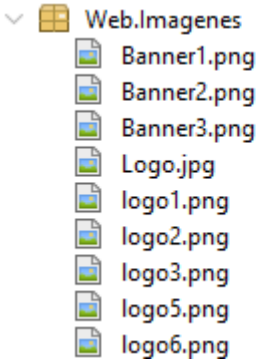
01/11
Hoy terminé la conexión del servidor web que había dejado el miércoles. Ahora sí se pueden ver las imágenes que era el principal problema, junto a lo del css.
Solo faltaría la parte de SQL.



Estos son los paquetes con los archivos de la página html que tenemos y el paquete de servidor con la conexión.



Y este es el paquete con las imágenes.



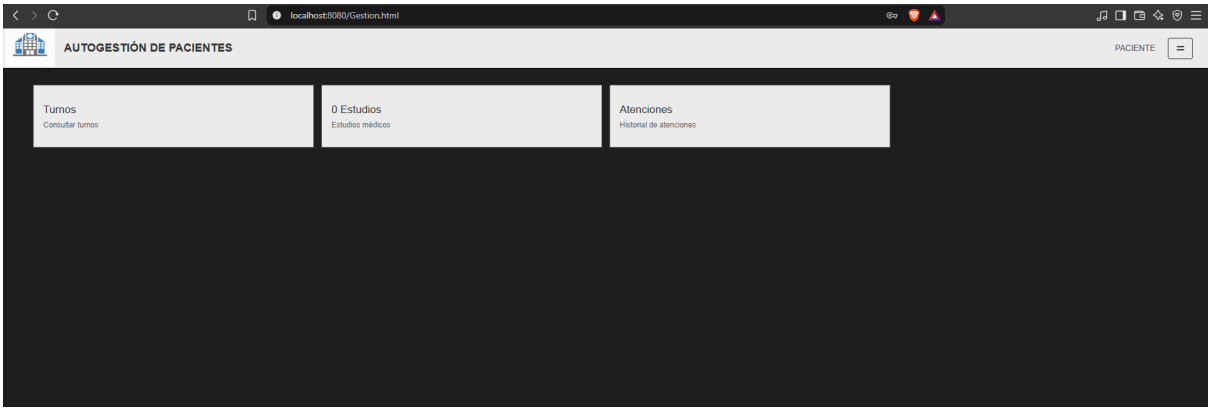
Al ejecutar tira esos “errores” pero se ejecuta la página igual. En si no son errores son avisos que dicen que el modelo de java está configurado en Java 8 y nuestro JDK es más nuevo.

```
Compiling 1 source file to C:\Users\thech\Downloads\Clinica_Privada_Servidor_Web\Clinica_Privada_Servidor_Web\build\classes
warning: [options] bootstrap class path is not set in conjunction with -source 8
  not setting the bootstrap class path may lead to class files that cannot run on JDK 8
    --release 8 is recommended instead of -source 8 -target 1.8 because it sets the bootstrap class path automatically
warning: [options] source value 8 is obsolete and will be removed in a future release
warning: [options] target value 8 is obsolete and will be removed in a future release
warning: [options] To suppress warnings about obsolete options, use -Xlint:-options.
4 warnings
compile:
run:
Servidor Java corriendo en http://localhost:8080/
Sirviendo archivos desde: C:\Users\thech\Downloads\Clinica_Privada_Servidor_Web\Clinica_Privada_Servidor_Web\src\Web
```

Esto es lo que vería el usuario cuando ingresa a la página, es decir la página inicial (index).

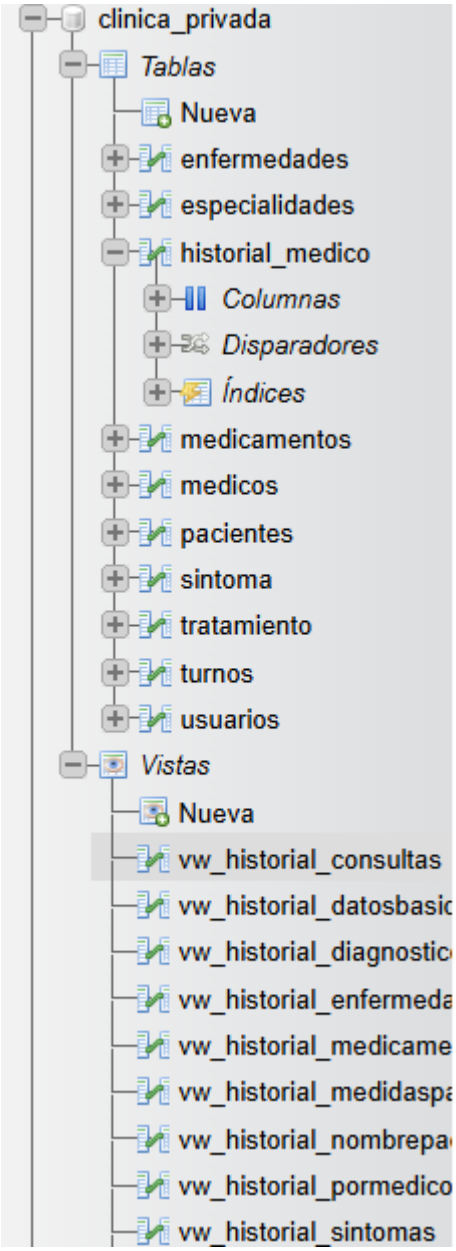


Esta es la parte de gestión, que antes no mostraba el logo de la clínica.



02/11
Hoy me puse a hacer las 10 vistas de historial médico y usuarios, además de sus 10 triggers.

Esta es la base de datos que subí en el servidor php para comprobar los triggers y las vistas.



Esta es la imagen de que las vistas de historial_medico no me dieron algún error por el momento.



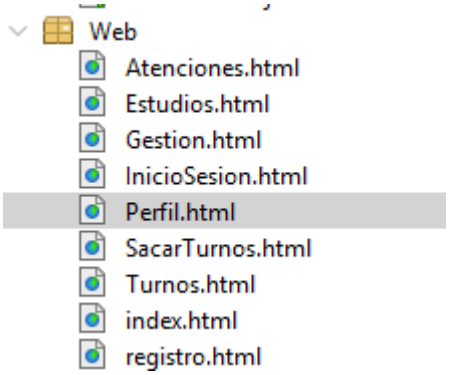
A continuación voy a adjuntar una imagen de una de las vistas hechas.

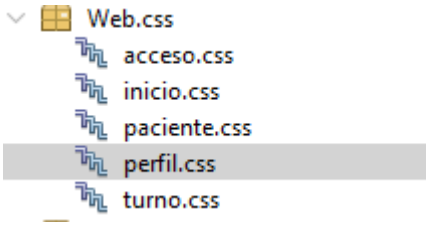


03/11
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

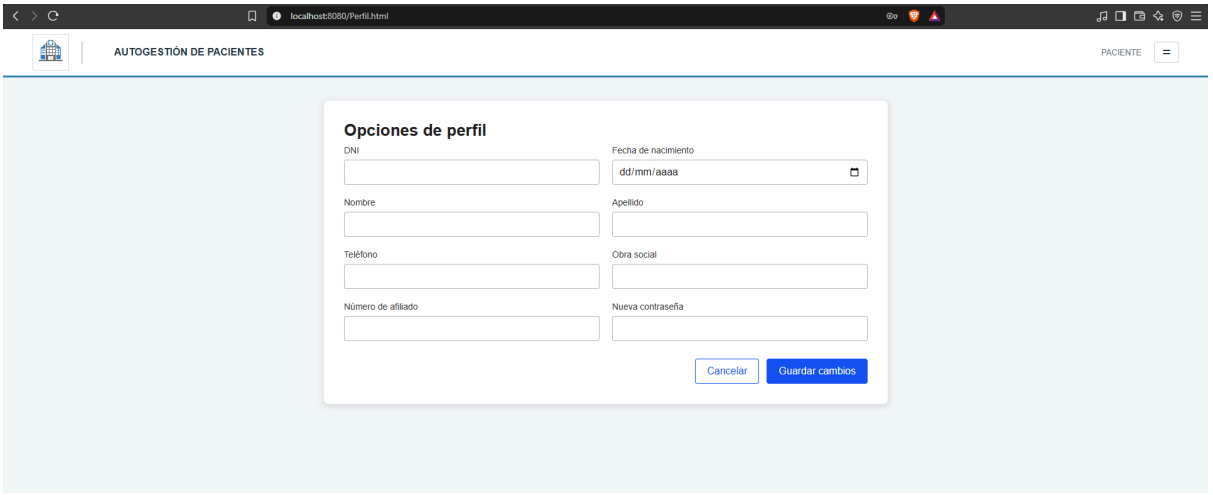
04/11
En la tarde me puse a agregar la nueva página que crearon, que era perfil, a la conexión de Java. También agregue su css de perfil.

A continuación adjunto imágenes.





Así se ve en la página ejecutada desde java.



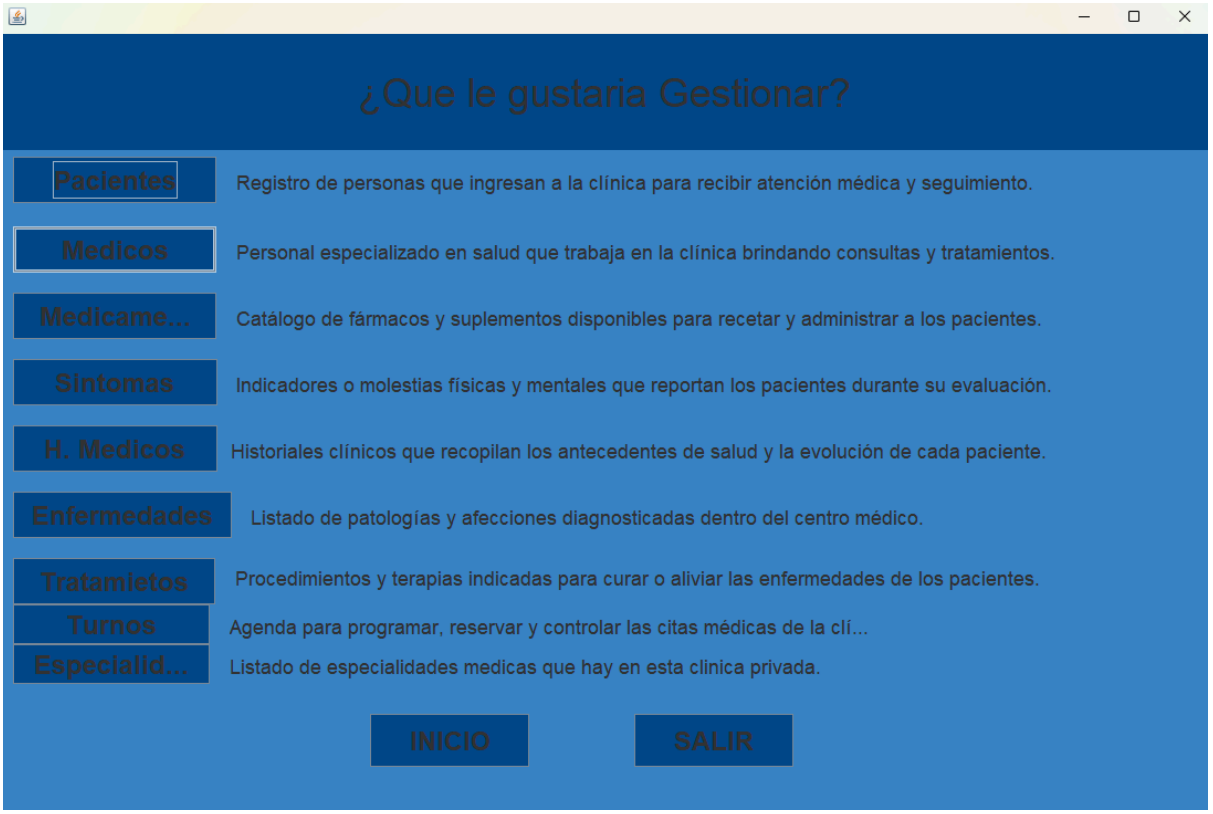
05/11
Este día no se realizó ninguna tarea/actividad sobre el proyecto.

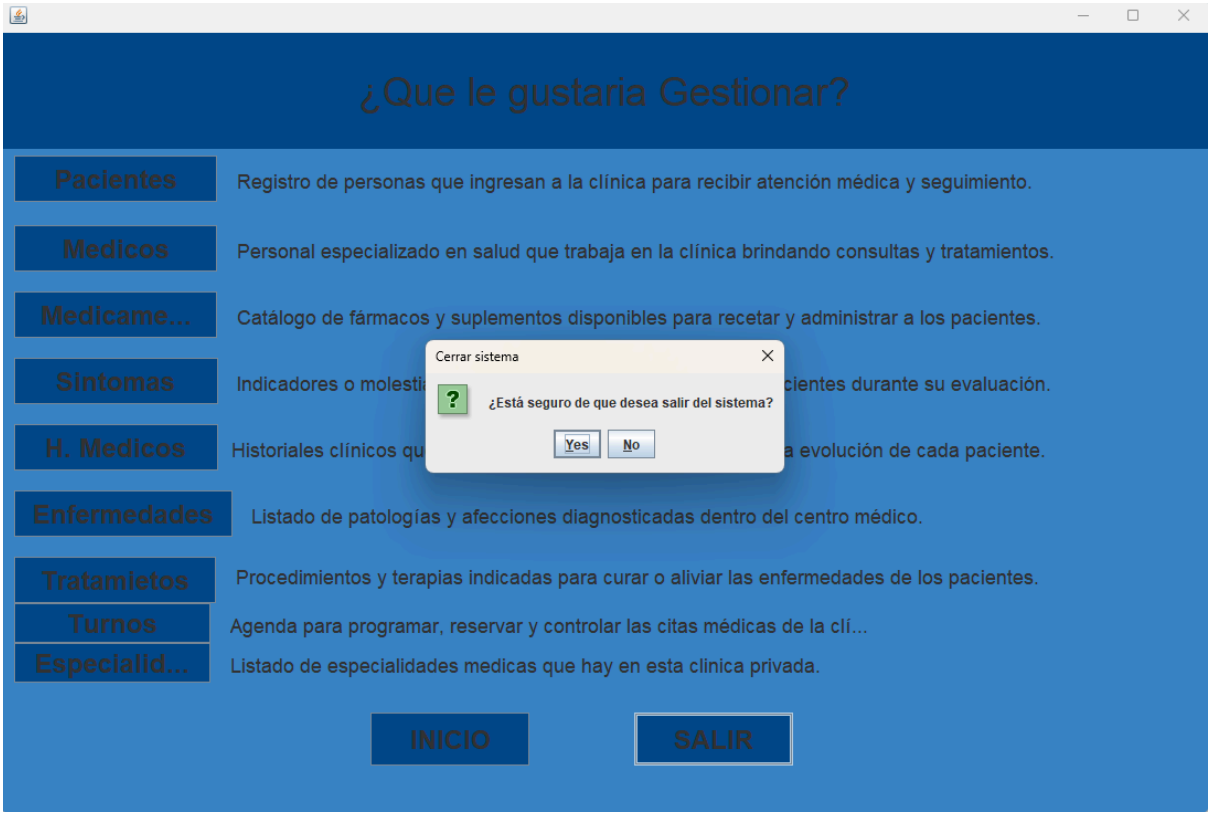
06/11
En el día de hoy con Matías Basto nos pusimos a testear la app de Java para ver que funcionaba y que no. Los resultados que obtuvimos del testeo fueron los siguientes:

- Falta agregar un botón de finalizar la aplicación.
- Carteles de Turno, y un cartel en enfermedad de el atributo que usa buscar.
- Y botón volver en la parte de inicio y registrarse.
- Mensajes de editar en algunas clases como gestion medico,

- Agregar los procedimientos de especialidades para que se pueda mostrar la tabla de enfermedades.
- Triggers en JAVA

Aca les muestro como quedo el boton de “SALIR”, que finaliza la aplicacion por completo, ademas tira un mensaje que te pregunta si estás seguro.





Este es el código del botón salir, con el mensaje de confirmación para finalizar la aplicación por completo.

```
private void btnSalirActionPerformed(java.awt.event.ActionEvent evt) {
int respuesta = JOptionPane.showConfirmDialog(
    this,
    "¿Está seguro de que desea salir del sistema?",
    "Cerrar sistema",
    JOptionPane.YES_NO_OPTION
);

if (respuesta == JOptionPane.YES_OPTION) {
    System.exit(0);
}
}
```

En la parte de editar turno de GestionTurno no dejaba editar, y la consola mostraba el siguiente error.

```
Fila seleccionada: 5
ID seleccionado: 7
ERROR No value specified for parameter 2
```

Era porque el 1 estaba repetido en la parte de actualizar de la clase TurnoDAO.

```java
public void actualizar(Turno turno) {
    String consulta_sql = "call Turnos_Actualizar(?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";

    try {
        Connection conexion = Conexion.conectar();
        CallableStatement sentencia_sql = conexion.prepareCall(consulta_sql);


        sentencia_sql.setInt(1, turno.getIdTurno());
        sentencia_sql.setString(1, turno.getCodigo_Paciente());
        sentencia_sql.setString(3, turno.getNombre_completoMedico());
        sentencia_sql.setString(4, turno.getFecha_Hora());
        sentencia_sql.setString(5, turno.getMatricula_medico());
        sentencia_sql.setString(6, turno.getEspecialidad());
        sentencia_sql.setInt(7, turno.getSala());
        sentencia_sql.setString(8, turno.getEstado());
        sentencia_sql.setString(9, turno.getMotivoConsulta());
        sentencia_sql.setString(10, turno.getTipoTurno());

        sentencia_sql.executeUpdate();
        System.out.println("Datos del turno actualizados exitosamente");
    } catch (SQLException ex) {
        System.out.println("ERROR " + ex.getMessage());
```

Así quedó después de la modificación.

```java
public void actualizar(Turno turno) {
    String consulta_sql = "call Turnos_Actualizar(?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";

    try {
        Connection conexion = Conexion.conectar();
        CallableStatement sentencia_sql = conexion.prepareCall(consulta_sql);


        sentencia_sql.setInt(1, turno.getIdTurno());
        sentencia_sql.setString(2, turno.getCodigo_Paciente());
        sentencia_sql.setString(3, turno.getNombre_completoMedico());
        sentencia_sql.setString(4, turno.getFecha_Hora());
        sentencia_sql.setString(5, turno.getMatricula_medico());
        sentencia_sql.setString(6, turno.getEspecialidad());
        sentencia_sql.setInt(7, turno.getSala());
        sentencia_sql.setString(8, turno.getEstado());
        sentencia_sql.setString(9, turno.getMotivoConsulta());
        sentencia_sql.setString(10, turno.getTipoTurno());
```

La razón del porque no mostraba el mensaje de editar era porque lo mandaba por consola y no por un JOptionPane.

```java
Medico medico = new Medico(especialidad, numero_matricula, años_experiencia, nombre, apellido,

medico.setId_medico(id_medico);

MedicoDAO dao = new MedicoDAO();
dao.actualizar(medico);

System.out.println("Datos modificados correctamente.");
```

Lo modifique para que el mensaje fuera un JOptionPane como las demás clases.

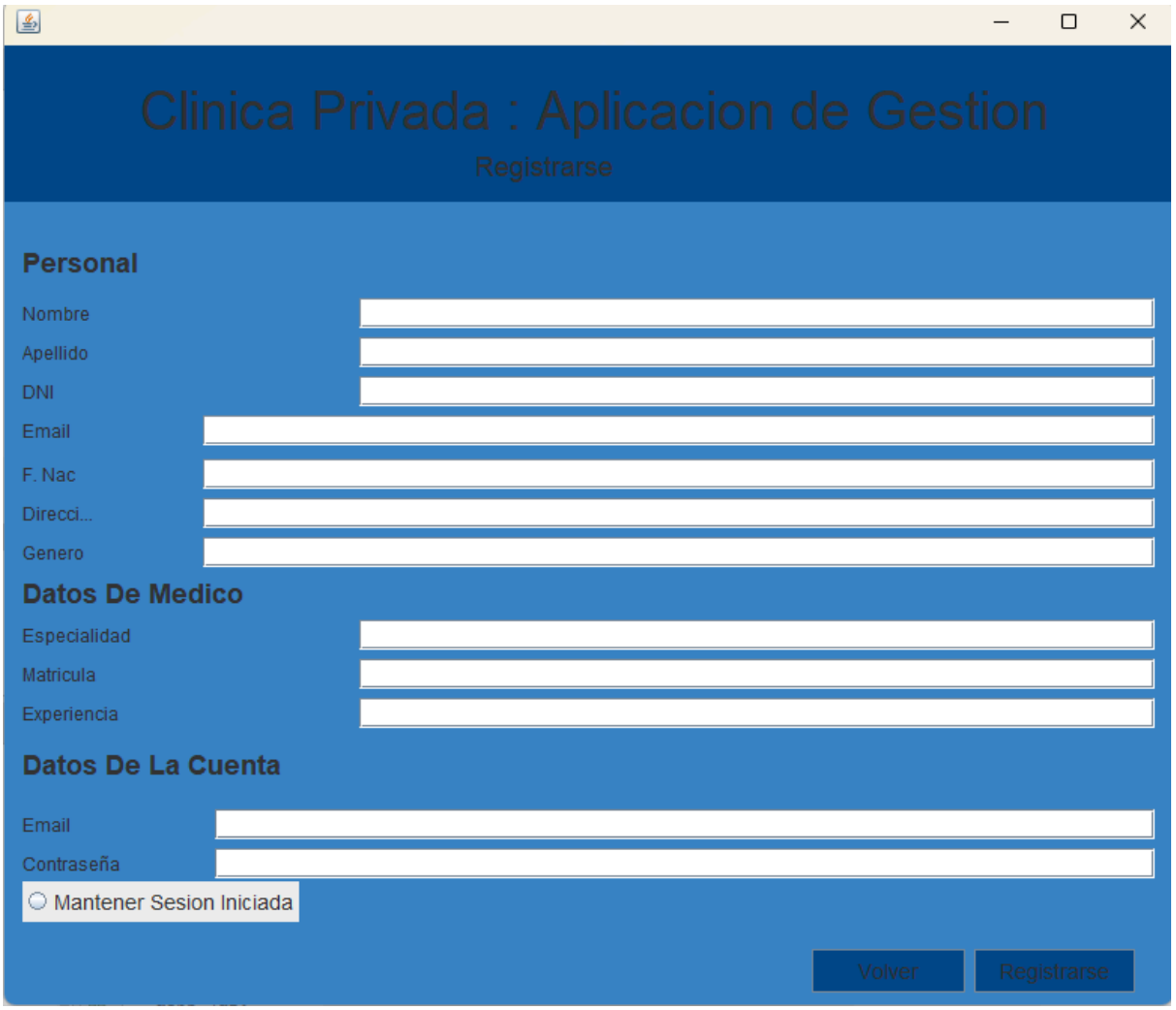
```
Medico medico = new Medico(especialidad, numero_matricula, años_experiencia, nombre, apellido, dni, email, fecha_nac

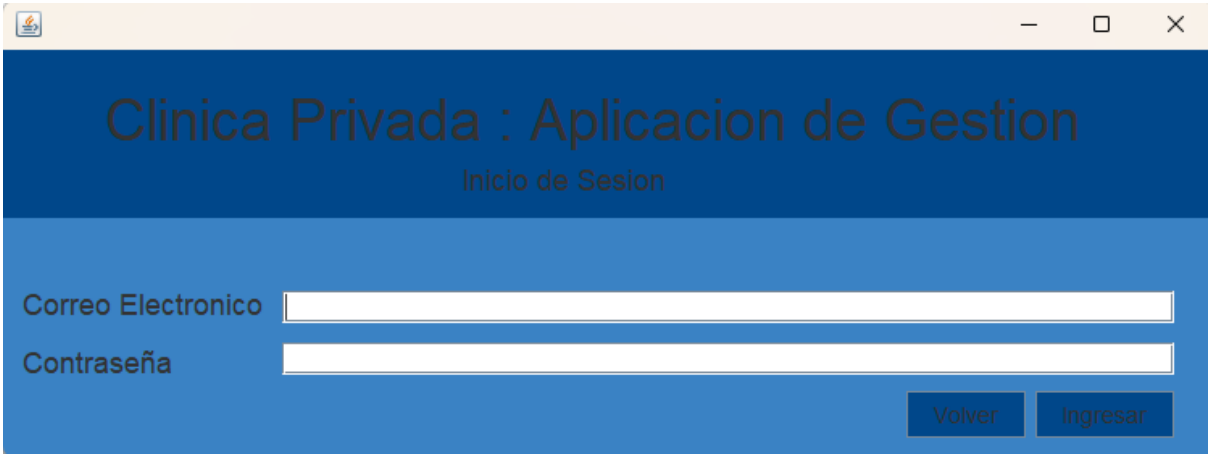
medico.setId_medico(id_medico);

MedicoDAO dao = new MedicoDAO();
dao.actualizar(medico);

JOptionPane.showMessageDialog(null, "Datos modificados correctamente.");
```

Y le agregue los botones de volver a “VentanaRegistrarse” y “VentanaIniciarSesion”, con su respectivo codigo.





Este código está en ambas ventanas, ya que ambas van a volver a la ventana de bienvenida.

```
private void btnVolverActionPerformed(java.awt.event.ActionEvent evt) {
    VentanaBienvenida ventanabienvenida = new VentanaBienvenida();
    ventanabienvenida.setVisible(true);
    ventanabienvenida.setLocationRelativeTo(null);
    this.setVisible(false);
}
```

Adjunto los procedimientos que faltaban para que la tabla de especialidades mostrara las especialidades que teníamos guardadas, además de poder ver las especialidades que se iban a agregar.

```sql
-- Especialidades

-- 1. mostrar todos
delimiter //
create procedure especialidades_mostrartodos()
begin
    select * from especialidades;
end //
delimiter ;

-- 2. buscar por nombre
delimiter //
create procedure especialidades_buscarpornombre(in p_nombre varchar(50))
begin
    select * from especialidades where nombre = p_nombre;
end //
delimiter ;

-- 3. registrar
delimiter //
create procedure especialidades_registrar(
    in p_nombre varchar(50),
    in p_descripcion text
)
begin
    insert into especialidades (nombre, descripcion)
    values (p_nombre, p_descripcion);
end //
delimiter ;

-- 4. actualizar
delimiter //
create procedure especialidades_actualizar(
    in p_id_especialidad int,
```

```sql
-- 4. actualizar
delimiter //
create procedure especialidades_actualizar(
    in p_id_especialidad int,
    in p_nombre varchar(50),
    in p_descripcion text
)
begin
    update especialidades
    set nombre = p_nombre,
        descripcion = p_descripcion
    where id_especialidad = p_id_especialidad;
end //
delimiter ;

-- 5. eliminar
delimiter //
create procedure especialidades_eliminar(in p_id_especialidad int)
begin
    delete from especialidades where id_especialidad = p_id_especialidad;
end //
delimiter ;
```